Введение в школу синтеза. Дискретная математика, логические элементы и микроэлектроника

Школа синтеза цифровых схем

ПРИ ПАРТНЕРСТВЕ

YADRO

2026

Школа синтеза цифровых схем



Владислав Терехин

- Аспирант 4 курса НИЯУ МИФИ на направлении: «Электронная компонентная базы микро- и наноэлектроники и квантовых устройств»
- Инженер 2 категории компании ООО «ИнноЦентр ВАО»
- Сотрудник кафедры микро- и наноэлектроники N°27 НИЯУ МИФИ
- Выпускник Школы синтеза 2025-2026

школа синтеза цифровых схем

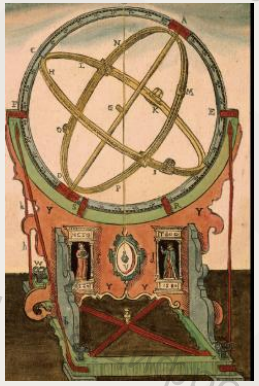
РАЗВИТИЕ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Развитие науки и технологий привело к тому, что математические задачи становились всё сложнее.

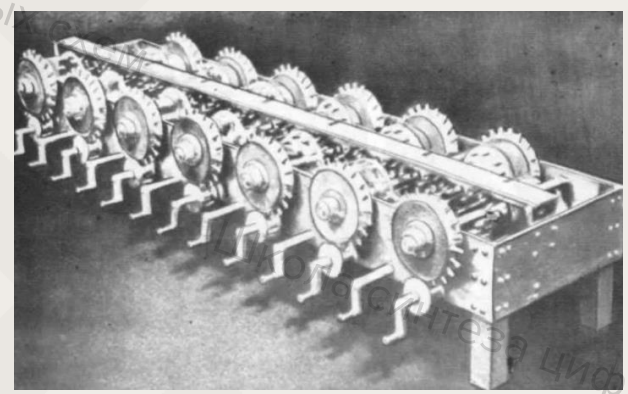
Автоматизация вычислений

100 лет до н.э. … 16 век н.э. 1673 год 1822 год

Армиллярная сфера. Определение координат небесных тел



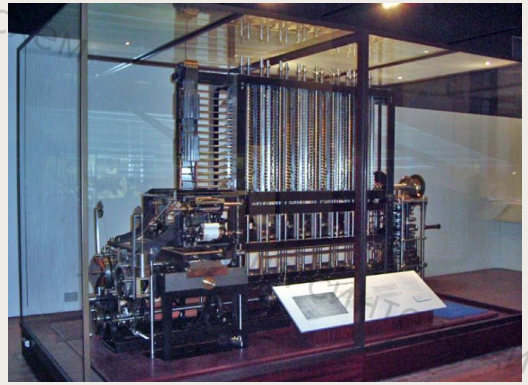
13-разрядное механическое счетное устройство Леонардо Да Винчи



Готфрид Лейбниц изобрел десятичную счетную машину, умеющую умножать и делить



Чарльз Бэббидж изобрел машину, вычисляющую значения многочленов до 6 степени.



Ранние вычислительные машины — история преодоления / Хабр

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

ЭВОЛЮЦИЯ ВЫЧИСЛИТЕЛЬНЫХ МАШИН

Вычислительная машина представляет из себя большое количество переключающихся элементов.

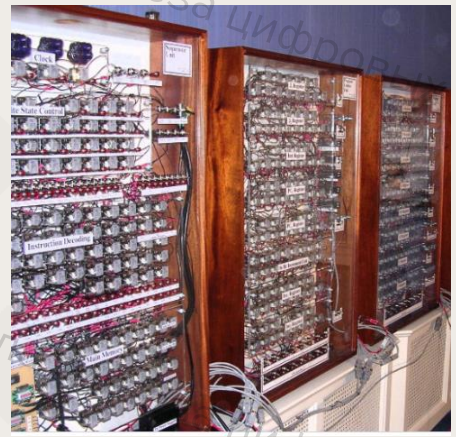
механические

шестеренки



электромеханические

реле



электронные

вакуумные лампы



электронные

транзисторы



A fistful of relays, или компьютер на электромагнитных реле. Часть 1 — АЛУ/ Хабр

РАЗВИТИЕ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Рост сложности электронных вычислительных машин требовал всё большего количества компонентов.

Необходимо было решить вопрос миниатюризации вычислительных устройств.

Этапы развития вычислительной техники на полупроводниках:

1947

ТРАНЗИСТОР

Bell Labs

полупроводниковый переключатель

1958–1959

ИНТЕГРАЛЬНАЯ СХЕМА

Килби • Нойс

элементы на одном кристалле

1959–1960

МОП-транзистор

Аталла • Канг

основа массовой интеграции

1971

INTEL 4004

≈ 2 300

транзисторов

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

МАСШТАБИРОВАНИЕ МИКРОЭЛЕКТРОНИКИ

$10^3 \rightarrow 10^6 \rightarrow 10^9 \rightarrow 10^{11}$



1971: 2 300 транзисторов

2020-е: 10^{10}–10^{11} транзисторов

ЗАКОН МУРА (1965)

Количество транзисторов на кристалле интегральной схемы удваивается примерно каждые 24 месяца

*историческая тенденция, не физический закон

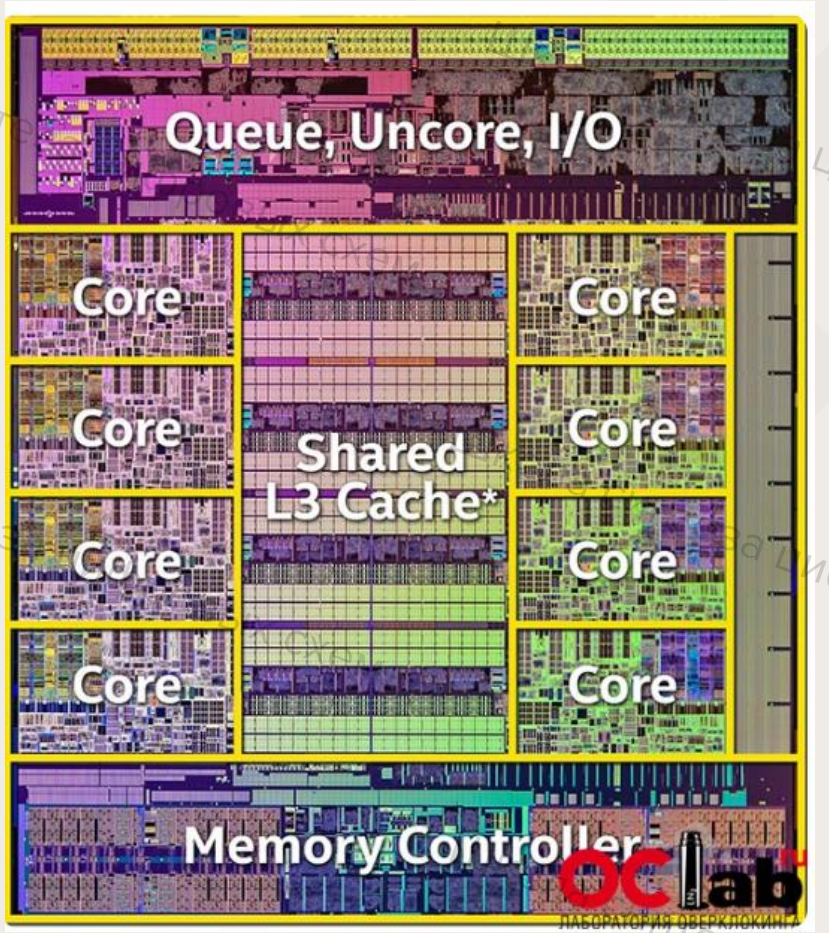
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

ЧТО ТАКОЕ МИКРОЭЛЕКТРОНИКА?

Микроэлектроника — раздел электроники, охватывающий исследования и разработку интегральных микросхем и принципов их применения.

Интегральная микросхема — совокупность, как правило, большого количества взаимосвязанных компонентов, изготовленная в едином технологическом цикле (т.е. одновременно), на одной и той же несущей конструкции — подложке.

Термин «интегральная схема» (ИС) отражает факт объединения (интеграции) отдельных компонентов в конструктивно единый прибор.



Фотка кристалла процессора **Intel Core i7-5960X** 2014 года.

школа синтеза цифровых схем

МИР БЕЗ ИНТЕГРАЛЬНЫХ МИКРОСХЕМ

Мысленный эксперимент: соберем современный чип из дискретных транзисторов

ВИДЕОКАРТА NVIDIA RTX 5090



92,2 млрд транзисторов;

Площадь : 750 кв. мм;
Цена: 170 000 рублей
Мощность: 600 Вт;
Тактовая частота: 2 ГГц

ТРАНЗИСТОР 2N7000



Площадь : 3,77 кв. мм;
Цена: 8 рублей;
Мощность: 0,5 Вт;
Тактовая частота: 67 МГц

МИР БЕЗ ИНТЕГРАЛЬНЫХ МИКРОСХЕМ

Расчёт для 92,2 млрд дискретных транзисторов при 8₽, 3 мм x 1,7 мм и 0,5 Вт на штуку

ЦЕНА

737,6 млрд. ₽

только транзисторы

ПЛОЩАДЬ

347 594 м²

квадратная плата со стороной ≈ 590 м

МОЩНОСТЬ

46,1 ГВт

Москва потребляет около 20 ГВт

БЫСТРОДЕЙСТВИЕ

67 МГц

максимальная тактовая частота

ВИДЕОКАРТА NVIDIA RTX 5090:

170 тысяч ₽	750 мм²	600 Вт	2 ГГц

Результат — не просто «меньше деталей», а другой масштаб и стоимость решения.

РАЗНЫЕ УРОВНИ АБСТРАКЦИИ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

01

Физический
Делаем транзистор

02

Схемотехнический
Делаем логические элементы на транзисторах

03

Функциональный
Превращаем напряжение в «0» и «1»

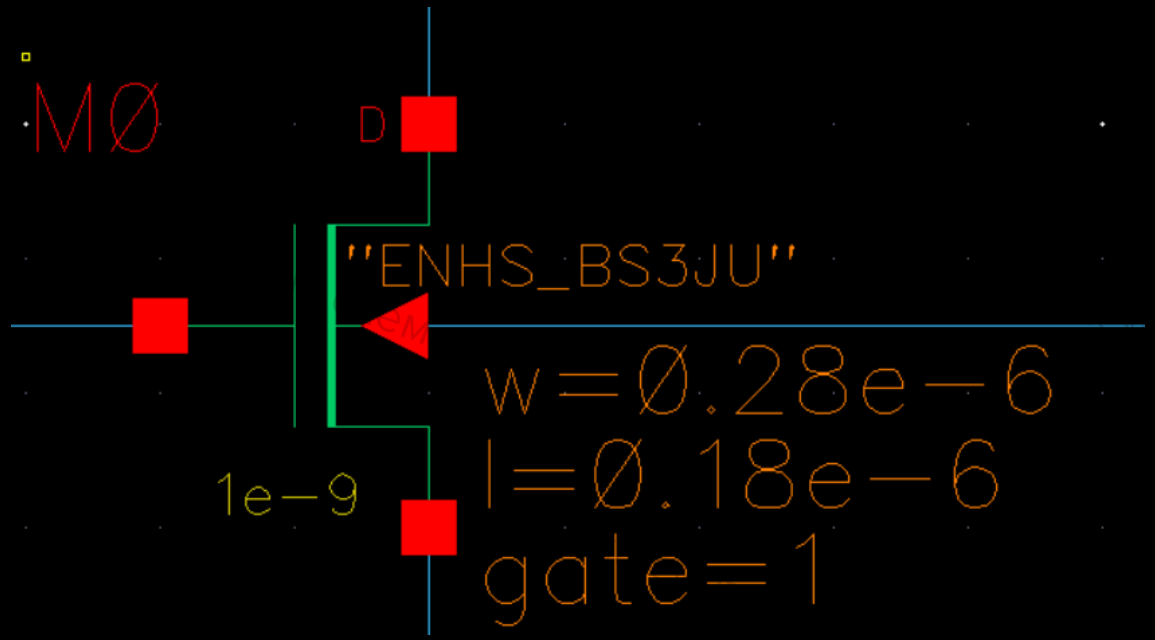
04

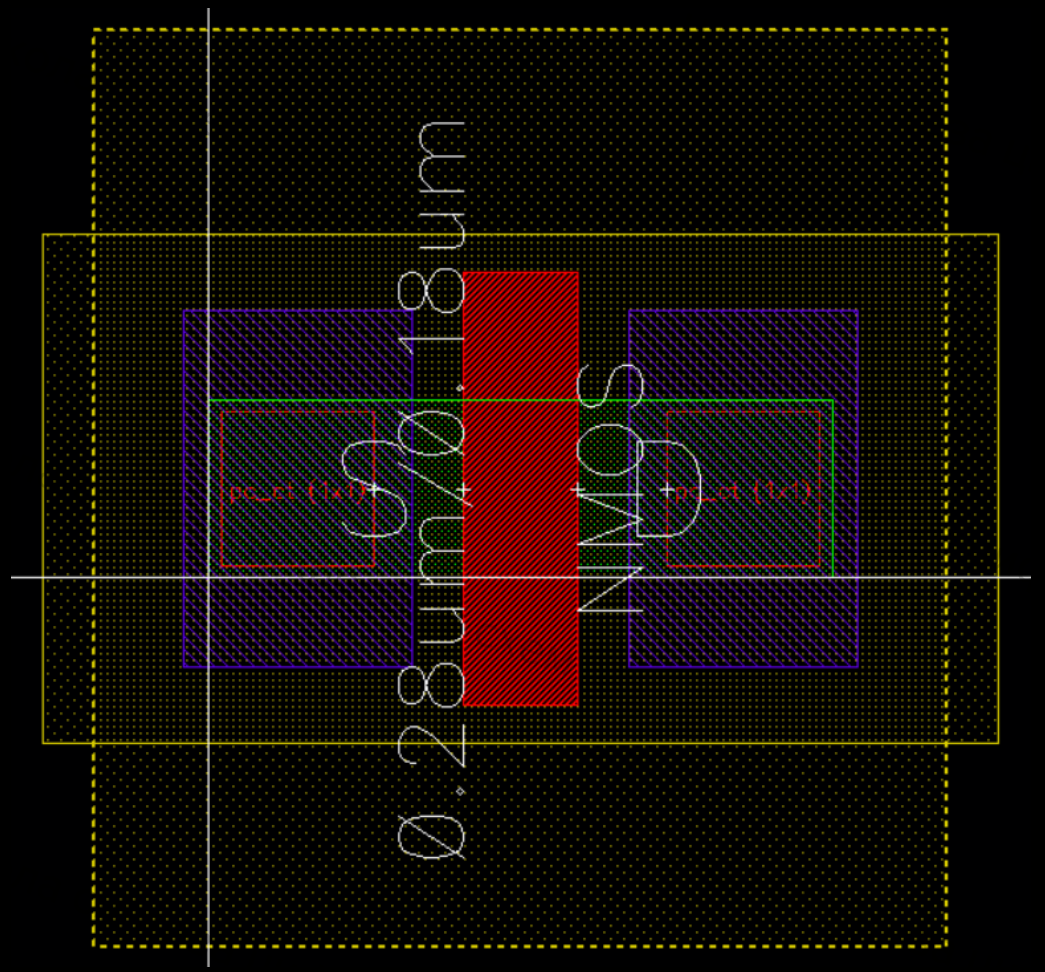
Микроархитектурный
Реализуем архитектурные требования к устройству на функциональных блоках

05

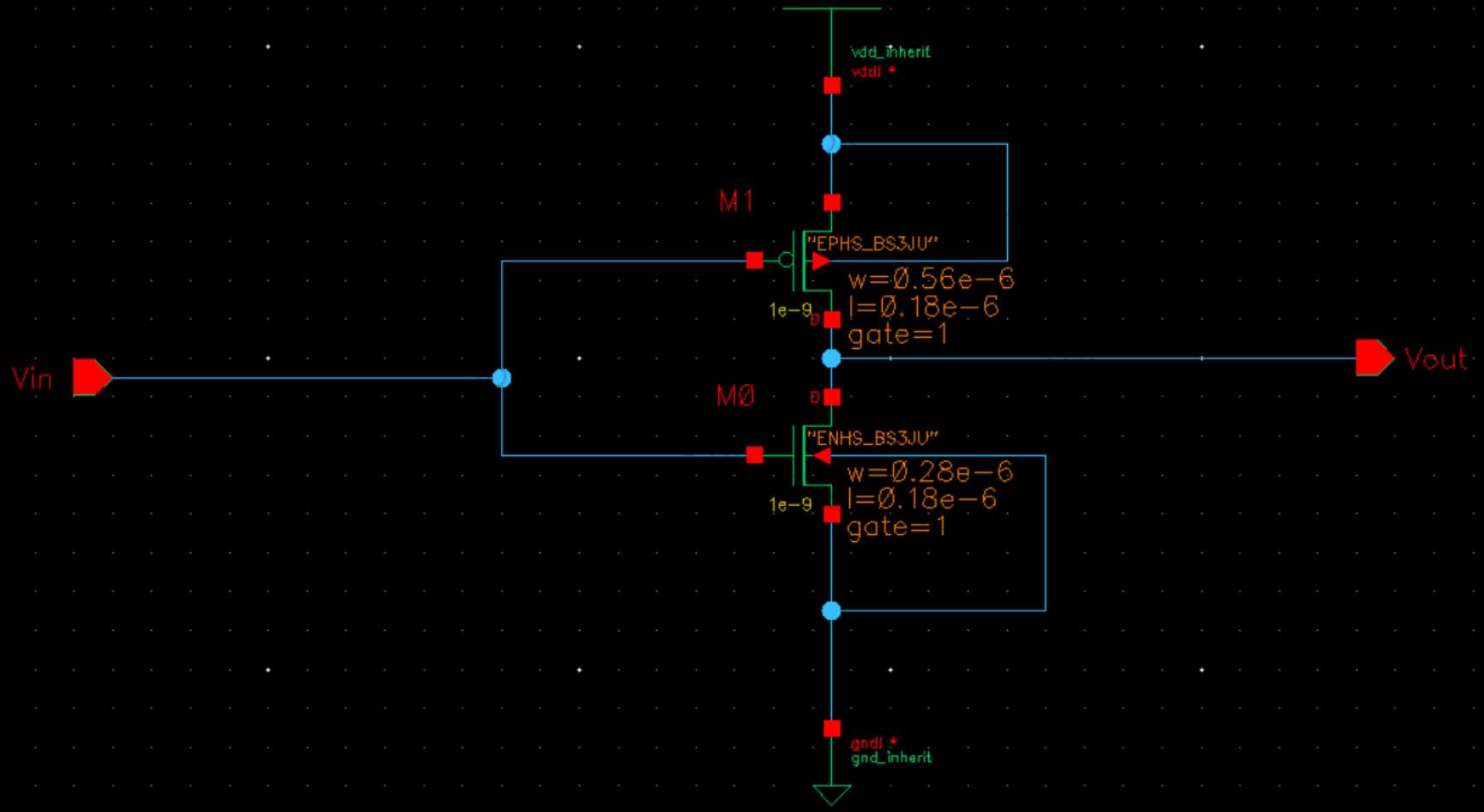
Архитектурный
Узнаём, как видит устройство программист

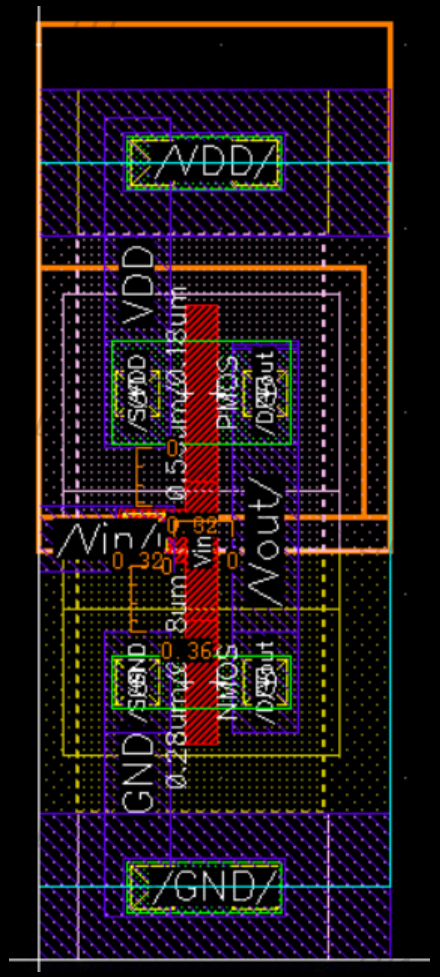
ФИЗИЧЕСКИЙ (ТРАНЗИСТОРНЫЙ) УРОВЕНЬ АБСТРАКЦИИ





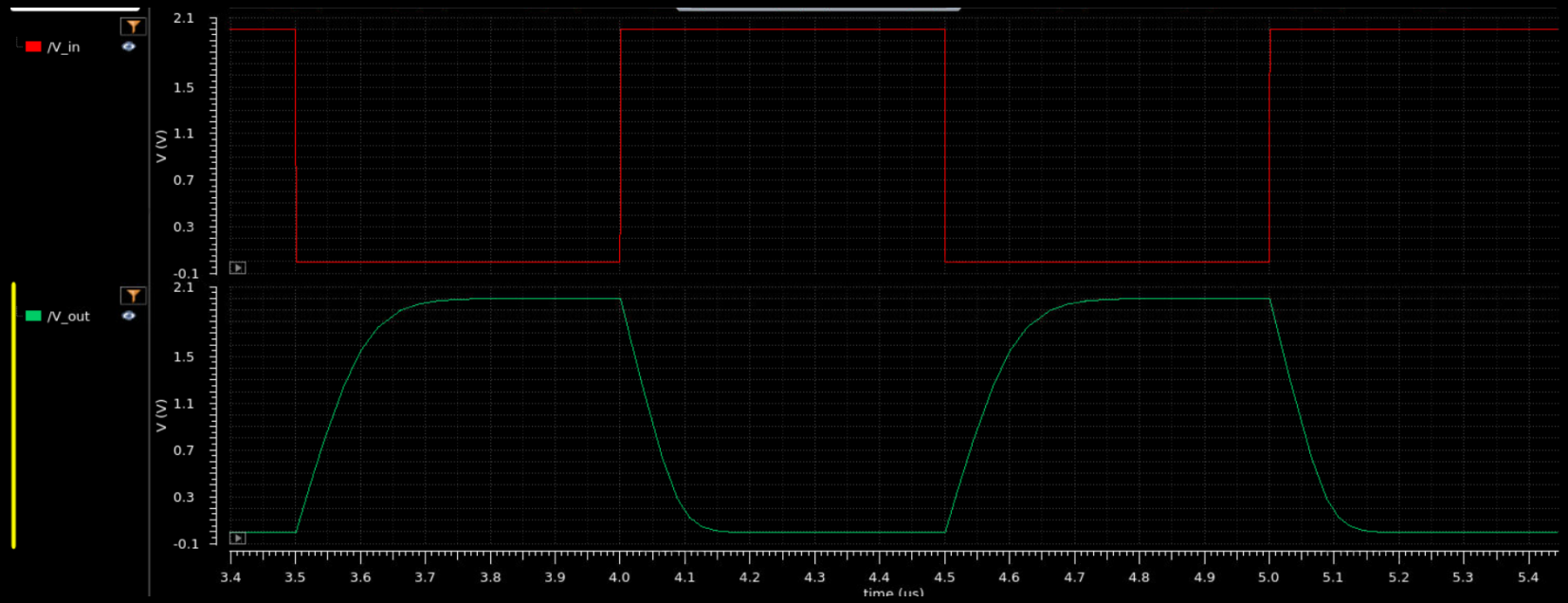
СХЕМОТЕХНИЧЕСКИЙ УРОВЕНЬ АБСТРАКЦИИ

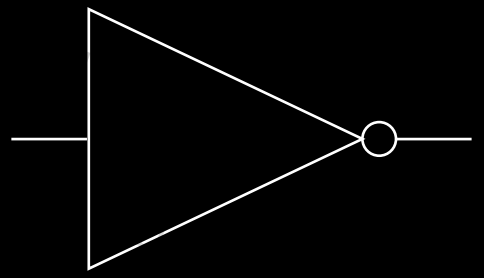




школа синтеза цифровых схем

ФУНКЦИОНАЛЬНЫЙ УРОВЕНЬ АБСТРАКЦИИ

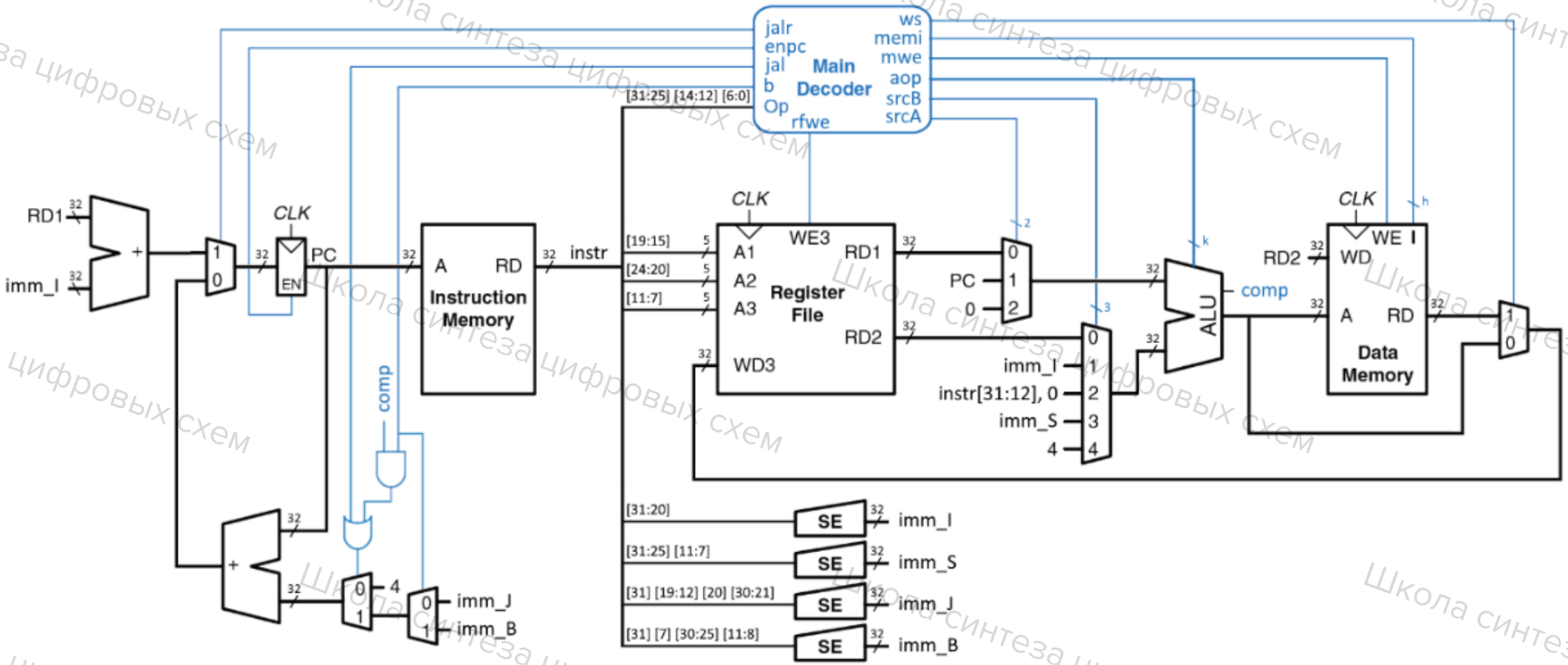




in	Out = ~in
0	1
1	0

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

МИКРОАРХИТЕКТУРНЫЙ УРОВЕНЬ АБСТРАКЦИИ



ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

АРХИТЕКТУРНЫЙ УРОВЕНЬ АБСТРАКЦИИ

Instr	Название	Функция	Описание	Формат	opcode	func3	func7	Пример использования
add	**ADD**ition	Сложение	rd = rs1 + rs2	R	0110011	0x0	0x00	
sub	**SUB**traction	Вычитание	rd = rs1 – rs2			0x0	0x20	
xor	e**X**clusive **OR**	Побитовое Исключающее ИЛИ	rd = rs1 ^ rs2			0x4	0x00	
or	**OR**	Побитовое ИЛИ	rd = rs1 | rs2			0x6	0x00	op rd, rs1, rs2
and	**AND**	Побитовое И	rd = rs1 & rs2			0x7	0x00	
sll	**S**hift **L**eft **L**ogical	Логический сдвиг влево	rd = rs1 << rs2			0x1	0x00	**xor** x2, x5, x6
srl	**S**hift **R**ight **L**ogical	Логический сдвиг вправо	rd = rs1 >> rs2			0x5	0x00	**sll** x7, x11, x12
sra	**S**hift **R**ight **A**rithmetic	Арифметический сдвиг вправо	rd = rs1 >>> rs2			0x5	0x20	
slt	**S**et **L**ess **T**han	Результат сравнения A < B	rd = (rs1 < rs2) ? 1 : 0			0x2	0x00	
sltu	**S**et **L**ess **T**han **U**nsigned	Беззнаковое сравнение A < B	rd = (rs1 < rs2) ? 1 : 0			0x3	0x00	
addi	**ADD**ition **I**mmediate	Сложение с константой	rd = rs1 + imm	I	0010011	0x0	–	
xori	e**X**clusive **OR** **I**mmediate	Побитовое Исключающее ИЛИ с константой	rd = rs1 ^ imm			0x4		
ori	**OR** **I**mmediate	Побитовое ИЛИ с константой	rd = rs1 | imm			0x6		op rd, rs1, imm
andi	**AND** **I**mmediate	Побитовое И с константой	rd = rs1 & imm			0x7		
slli	**S**hift **L**eft **L**ogical **I**mmediate	Логический сдвиг влево	rd = rs1 << imm	I*		0x1	0x00	**addi** x6, x3, -12
srli	**S**hift **R**ight **L**ogical **I**mmediate	Логический сдвиг вправо	rd = rs1 >> imm			0x5	0x00	**ori** x3, x1, 0x8F
srai	**S**hift **R**ight **A**rithmetic **I**mmediate	Арифметический сдвиг вправо	rd = rs1 >>> imm			0x5	0x20	
slti	**S**et **L**ess **T**hen **I**mmediate	Результат сравнения A < B	rd = (rs1 < imm) ? 1 : 0	I		0x2	–	
sltiu	**S**et **L**ess **T**hen **I**mmediate **U**nsigned	Беззнаковое сравнение A < B	rd = (rs1 < imm) ? 1 : 0			0x3		
lb	**L**oad **B**yte	Загрузить байт из памяти	rd = SE(Mem[rs1 + imm][7:0])	I	0000011	0x0	–	
lh	**L**oad **H**alf	Загрузить полуслово из памяти	rd = SE(Mem[rs1 + imm][15:0])			0x1		op rd, imm(rs1)
lw	**L**oad **W**ord	Загрузить слово из памяти	rd = SE(Mem[rs1 + imm][31:0])			0x2		
lbu	**L**oad **B**yte **U**nsigned	Загрузить беззнаковый байт из памяти	rd = Mem[rs1 + imm][7:0]			0x4		**lh** x1, 8(x5)
lhu	**L**oad **H**alf **U**nsigned	Загрузить беззнаковое полуслово из памяти	rd = Mem[rs1 + imm][15:0]			0x5		
sb	**S**tore **B**yte	Сохранить байт в память	Mem[rs1 + imm][7:0] = rs2[7:0]	S	0100011	0x0	–	op rs2, imm(rs1)
sh	**S**tore **H**alf	Сохранить полуслово в память	Mem[rs1 + imm][15:0] = rs2[15:0]			0x1		
sw	**S**tore **W**ord	Сохранить слово в память	Mem[rs1 + imm][31:0] = rs2[31:0]			0x2		**sw** x1, 0xCF(x12)
beq	**B**ranch if **E**qual	Перейти, если A == B	if (rs1 == rs2) PC += imm	B	1100011	0x0	–	
bne	**B**ranch if **N**ot **E**qual	Перейти, если A != B	if (rs1 != rs2) PC += imm			0x1		comp rs1, rs2, imm
blt	**B**ranch if **L**ess **T**han	Перейти, если A < B	if (rs1 < rs2) PC += imm			0x4		
bge	**B**ranch if **G**reater or **E**qual	Перейти, если A >= B	if (rs1 >= rs2) PC += imm			0x5		**beq** x8, x9, offset
bltu	**B**ranch if **L**ess **T**han **U**nsigned	Перейти, если A < B беззнаковое	if (rs1 < rs2) PC += imm			0x6		**bltu** x20, x21, 0xFC
bgeu	**B**ranch if **G**reater or **E**qual **U**nsigned	Перейти, если A >= B беззнаковое	if (rs1 >= rs2) PC += imm			0x7		
jal	**J**ump **A**nd **L**ink	Переход с сохранением адреса возврата	rd = PC + 4; PC += imm	J	1101111	–	–	**jal** x1, offset
jalr	**J**ump **A**nd **L**ink **R**egister	Переход по регистру с сохранением адреса возврата	rd = PC + 4; PC = rs1 + imm	I	1100111	0x0		**jalr** x1, 0(x5)
lui	**L**oad **U**pper **I**mmediate	Загрузить константу в сдвинутую на 12	rd = imm << 12	U	0110111	–	–	**lui** x3, 0xFFFFF
auipc	**A**dd **U**pper **I**mmediate to **PC**	Сохранить счетчик команд в сумме с константой << 12	rd = PC + (imm << 12)		0010111			**auipc** x2, 0x000FF
fence	**FENCE**	Синхронизация инструкций с потоками данных	Воспринимать как **nop**	I	0001111	0x0	–	**fence**
ecall	**E**nvironment **CALL**	Передача управления операционной системе	illegal_instr = 1	I**	1110011	0x0	–	**ecall**
ebreak	**E**nvironment **BREAK**	Передача управления отладчику						**ebreak**
mret	**M**achine mode **RET**URN	Возврат управления от обработчика прерывания/исключения	PC = mepc	I**	1110011	0x0	–	**mret**
csrrw	**C**ontrol **S**tatus **R**egister **R**ead **W**rite	Записать в регистровый файл значение **CS**-регистра. Обновить значение **CS**-регистра в соответствии с вызываемой инструкцией (см. описание).	csr = rs1; rd = csr	I	1110011	0x1	–	csrop rd, csr, rs1
csrrs	**C**ontrol **S**tatus **R**egister **R**ead **S**et		csr |= rs1; rd = csr			0x2		
csrrc	**C**ontrol **S**tatus **R**egister **R**ead **C**lear		csr &= ~rs1; rd = csr			0x3		
csrrwi	**C**ontrol **S**tatus **R**egister **R**ead **W**rite **I**mmediate		csr = imm; rd = csr			0x5		**csrrwi** x0, mie, 0x0FF
csrrsi	**C**ontrol **S**tatus **R**egister **R**ead **S**et **I**mmediate		csr |= imm; rd = csr			0x6		
csrrci	**C**ontrol **S**tatus **R**egister **R**ead **C**lear **I**mmediate		csr &= ~imm; rd = csr			0x7		

ОСНОВНЫЕ ЭТАПЫ РАЗРАБОТКИ ЧИПА

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

01

Спецификация

Описать замысел микросхемы на бумаге

02

Проектирование

Реализация замысла проекта путем создания его описания на уровне регистровой передачи (RTL)

03

Верификация

Подтвердить, что RTL-код соответствует первоначальным намерениям

04

Имплементация

Синтезировать RTL-код в стандартные 32-нм ячейки

Размещение и трассировка

05

Производство

Изготовление набора масок

Формирование каждого слоя на поверхности кремния

06

Отбраковка

Проверка микросхем перед использованием

RTL-ОПИСАНИЕ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

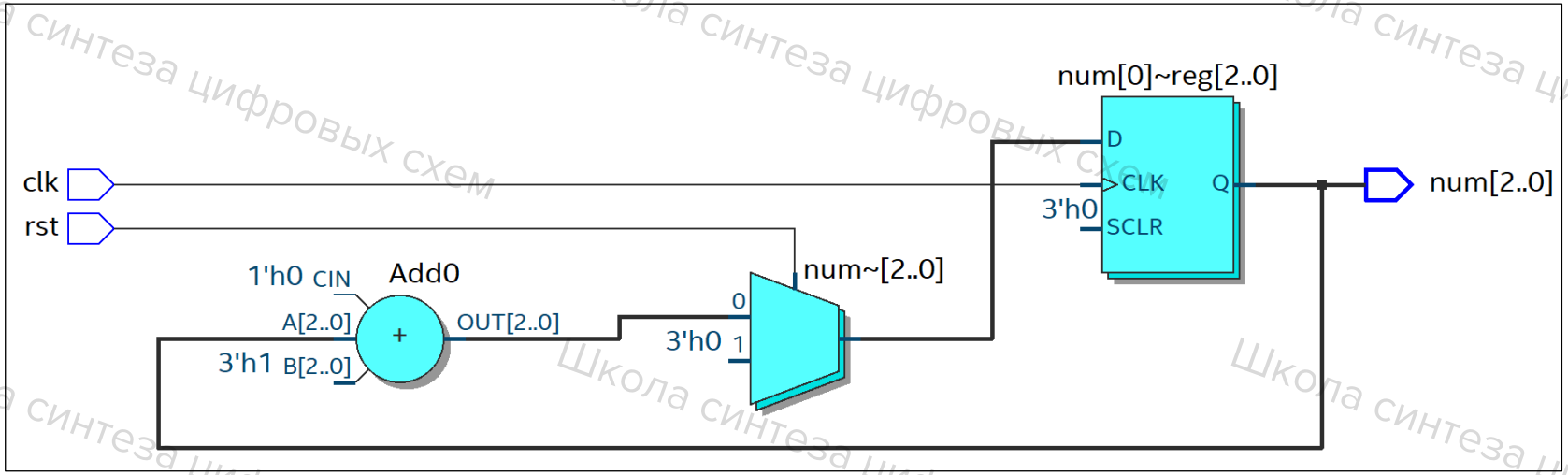
```
module counter_3_bit (
    input               clk,
    input               rst,
    output logic [2:0] num
);

always_ff @(posedge clk) begin
    if (rst) num <= '0;
    else num <= num + 1;
end

endmodule
```

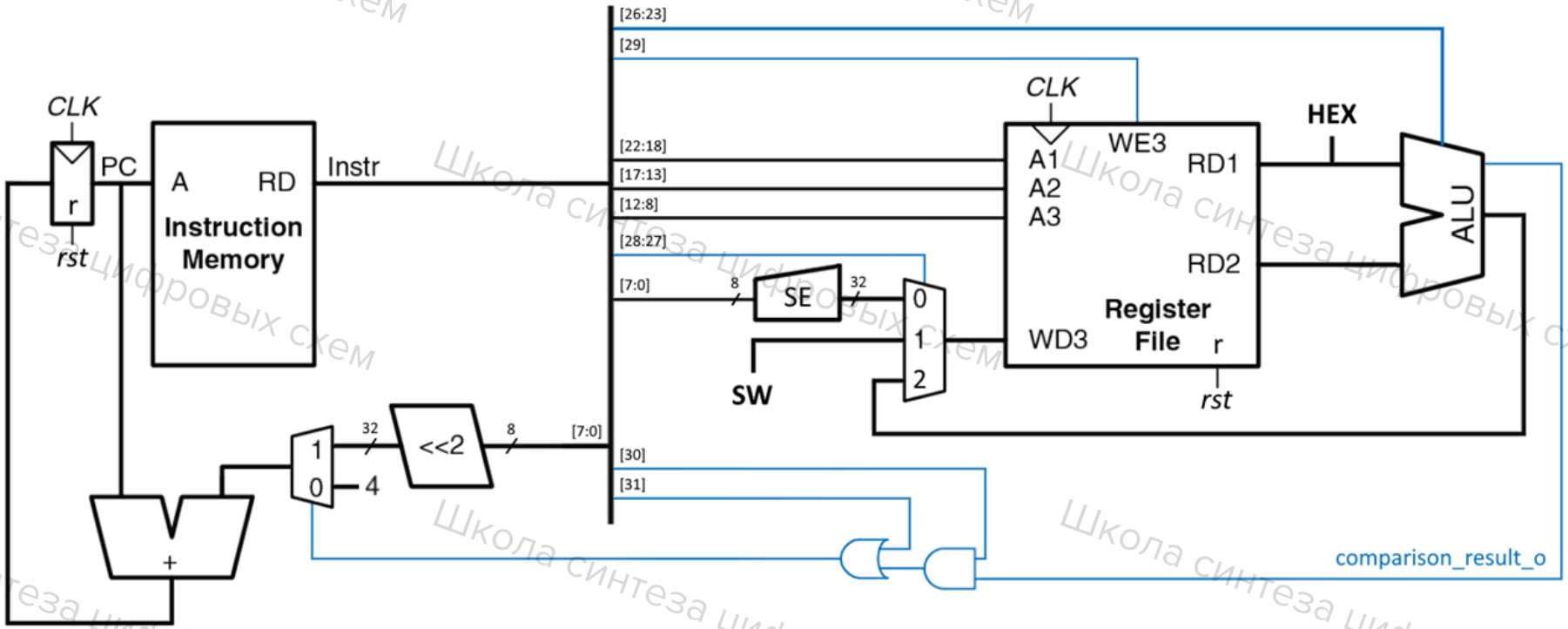
RTL
Register Transfer Level

Описание на уровне регистровых передач
Поведенческое описание схемы



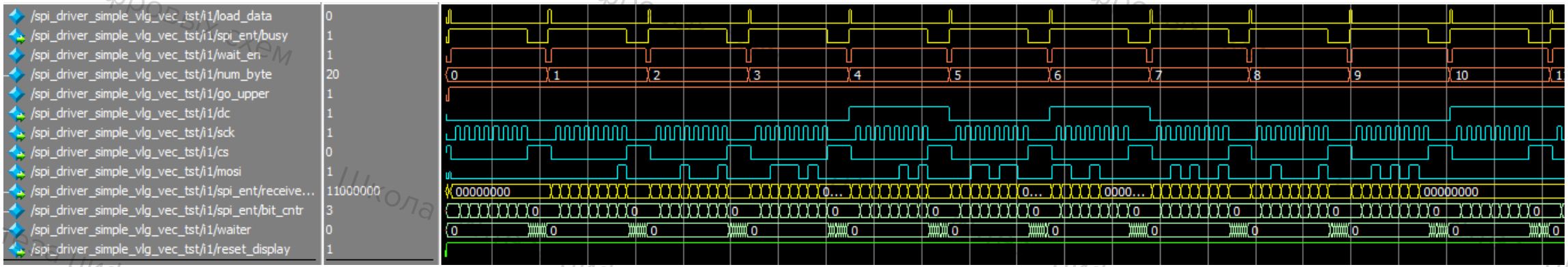
RTL-РАЗРАБОТЧИКИ

RTL-разработчики придумывают алгоритм (микроархитектуру), который реализует функциональные требования



ЧЕМ ЗАНИМАЕТСЯ ВЕРИФИКАТОР

Верификатор пишет проверочный код (testbench), который подает входные воздействия на входы схемы и смотрит на соответствие RTL-описания заявленной функциональности



ПОЧЕМУ ВЕРИФИКАТОРЫ ВАЖНЫ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

- Стоимость разработки комплекта фотошаблонов оценивается в **$2-3 млн**.
- **Дешевле** платить большому штату верификаторов, чем выкидывать миллионы долларов из-за ошибки в RTL

TSMC 28-нм высокопроизводительный HKMG КМОП

10 слоев меди, двойной оксид затвора (DGO), 55 масок, 300-мм пластины, Fab 14 (Тайвань)

Процесс производства	Единицы измерения	Значение
Стоимость чистой пластины	$/пластина	$125
Маржа foundry-производителя	$/пластина	$2,500
Себестоимость пластины (без сортировки)	$/пластина	$5,200
Стоимость сортировки пластин	$/пластина	$200
Итоговая себестоимость пластины	$/пластина	$5,400
Выход годных кристаллов	%	90
Всего кристаллов на пластине	—	2000
Годных кристаллов на пластине	—	1800
Стоимость одного годного отсортированного кристалла	$/кристалл	$3.00

Этап сборки и тестирования	Единицы измерения	Стоимость
Стоимость корпусирования	$/корпус	$0.12
Стоимость финального тестирования	$/устройство	$0.20
Общая стоимость устройства	**$/устройство**	**$3.32**

АКЦЕНТЫ ШКОЛЫ СИНТЕЗА

01
Спецификация
Описать замысел микросхемы на бумаге

02
Проектирование
Реализация замысла проекта путем создания его описания на уровне регистровой передачи (RTL)

03
Верификация
Подтвердить, что RTL-код соответствует первоначальным намерениям

04
Имплементация
Синтезировать RTL-код в стандартные 32-нм ячейки

Размещение и трассировка

05
Производство
Изготовление набора масок

Формирование каждого слоя на поверхности кремния

06
Отбраковка
Проверка микросхем перед использованием

АКЦЕНТЫ ШКОЛЫ СИНТЕЗА

01
Спецификация
Описать замысел микросхемы на бумаге

02
Проектирование
Реализация замысла проекта путем создания его описания на уровне регистровой передачи (RTL)

03
Верификация
Подтвердить, что RTL-код соответствует первоначальным намерениям

04
Имплементация
Синтезировать RTL-код в стандартные 32-нм ячейки

Размещение и трассировка

05
Производство
Изготовление набора масок

Формирование каждого слоя на поверхности кремния

06
Отбраковка
Проверка микросхем перед использованием

АКЦЕНТЫ ШКОЛЫ СИНТЕЗА

01
Спецификация
Описать замысел микросхемы на бумаге

02
Проектирование
Реализация замысла проекта путем создания его описания на уровне регистровой передачи (RTL)

03
Верификация
Подтвердить, что RTL-код соответствует первоначальным намерениям

04
Имплементация
Синтезировать RTL-код в стандартные 32-нм ячейки

Размещение и трассировка

05
Производство
Изготовление набора масок

Формирование каждого слоя на поверхности кремния

06
Отбраковка
Проверка микросхем перед использованием

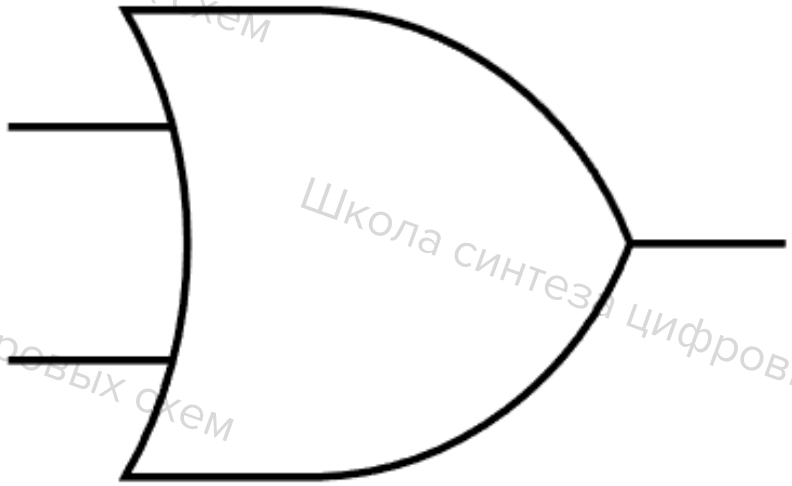
ЭЛЕМЕНТ «И»



a	b	F = a & b
0	0	0
0	1	0
1	0	0
1	1	1

Школа синтеза цифровых схем

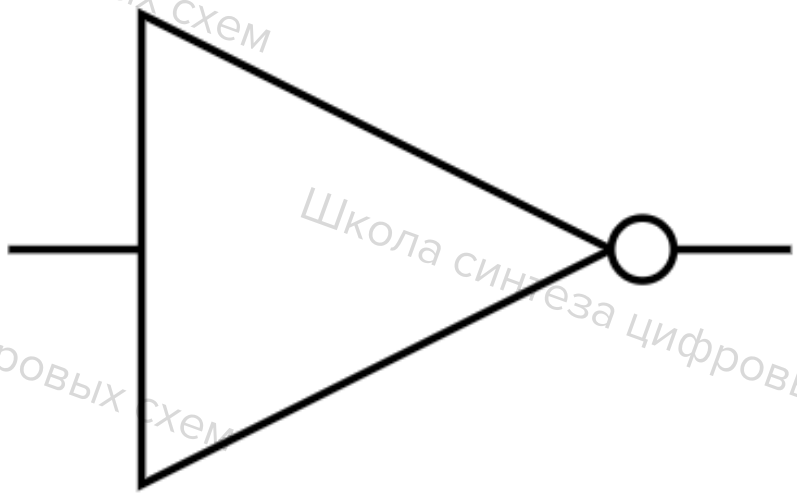
ЭЛЕМЕНТ «ИЛИ»



a	b	F = a | b
0	0	0
0	1	1
1	0	1
1	1	1

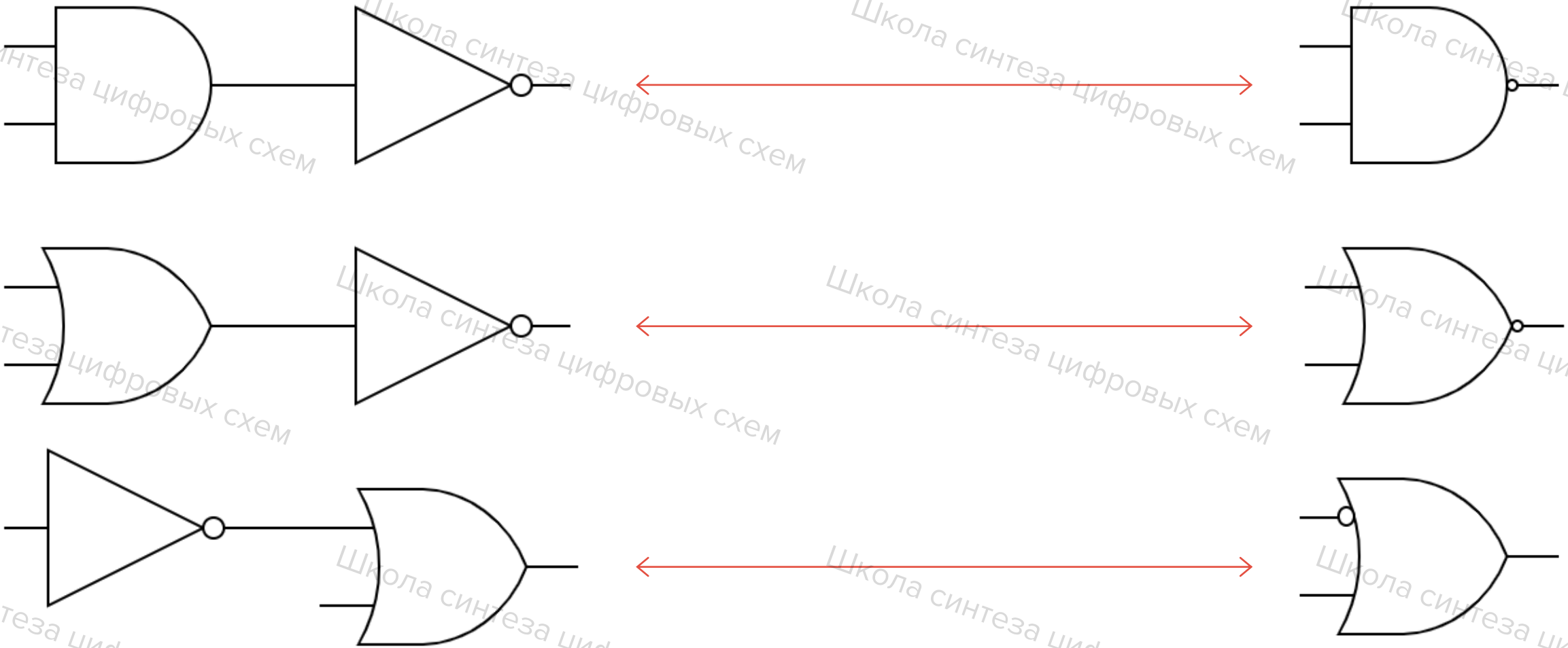
Школа синтеза цифровых схем

ИНВЕРТОР (ЭЛЕМЕНТ «НЕ»)



in	out = ~in
0	1
1	0

ОБОЗНАЧЕНИЕ ИНВЕРСИИ

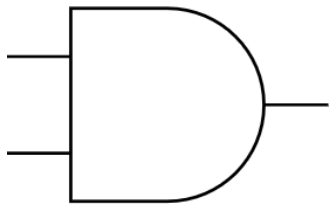


Школа синтеза цифровых схем

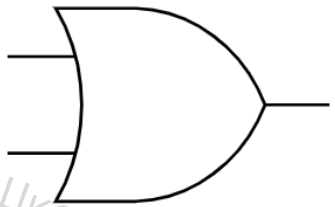
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

ПОСТРОЕНИЕ ЛЮБОЙ ФУНКЦИИ НА «И/ИЛИ/НЕ»

a	b	F = a ^ b
0	1	1
1	0	1



Детектирует «1»



Соединяет детекторы

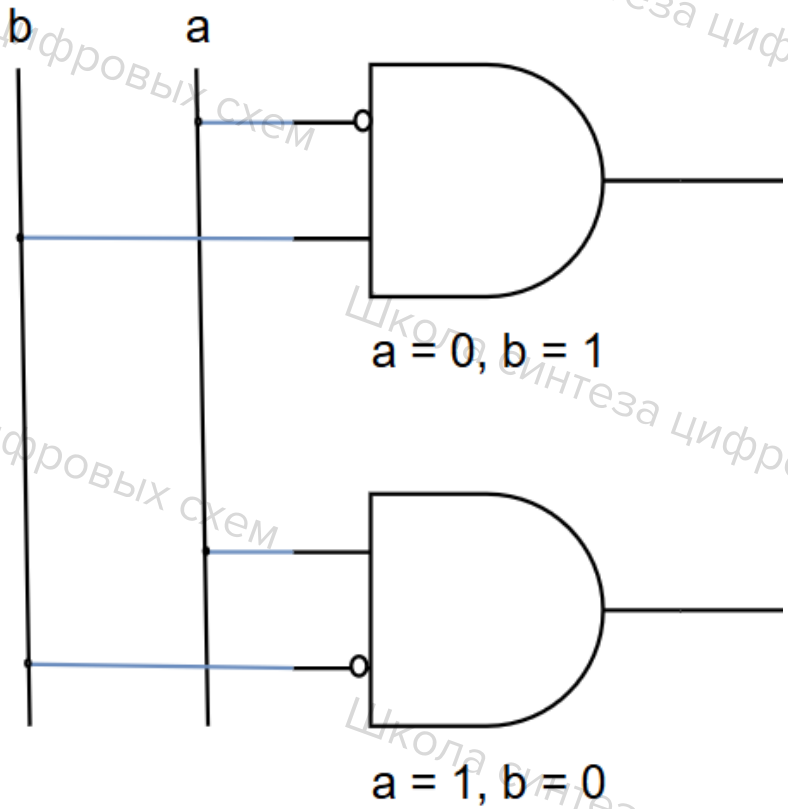
a	b	F = a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

Школа синтеза цифровых схем

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

ДЕТЕКТИРУЕМ ВХОДЫ, ПРИ КОТОРЫХ ФУНКЦИЯ F = 1

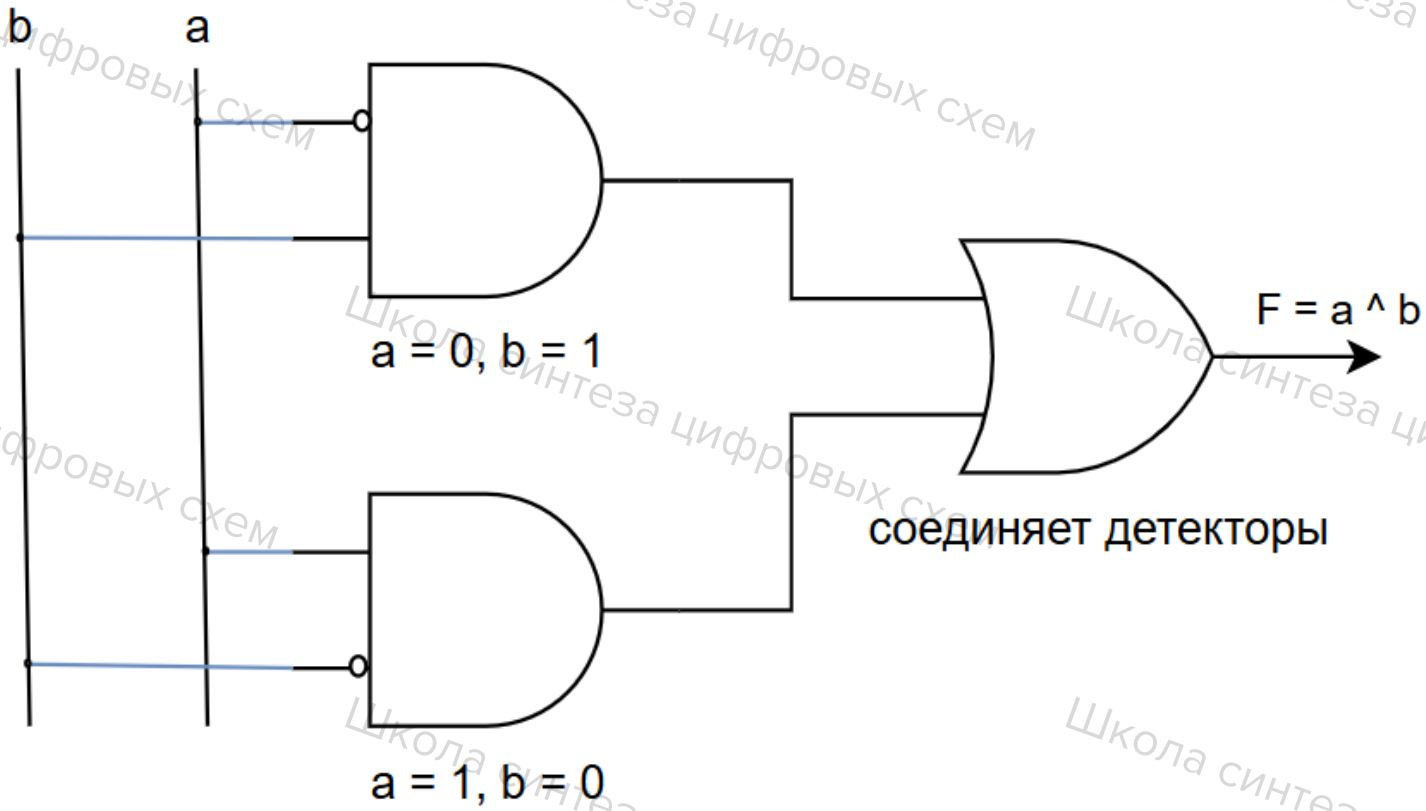
a	b	F = a ^ b
0	1	1
1	0	1



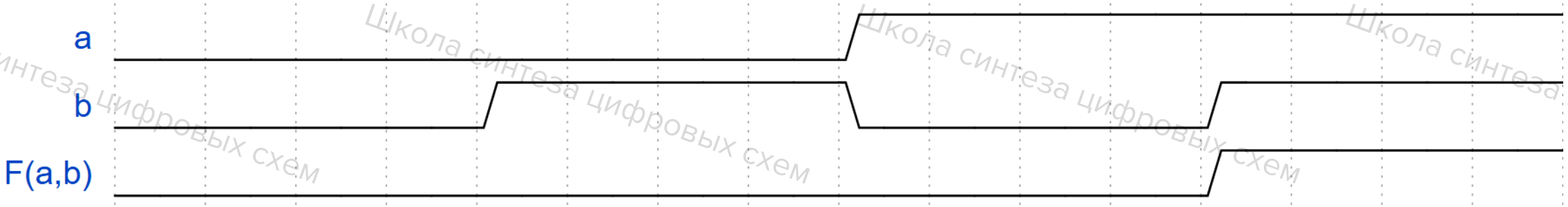
СОЕДИНЯЕМ ДЕТЕКТИРУЮЩИЕ «И»

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

a	b	F = a ^ b
0	0	0
0	1	1
1	0	1
1	1	0



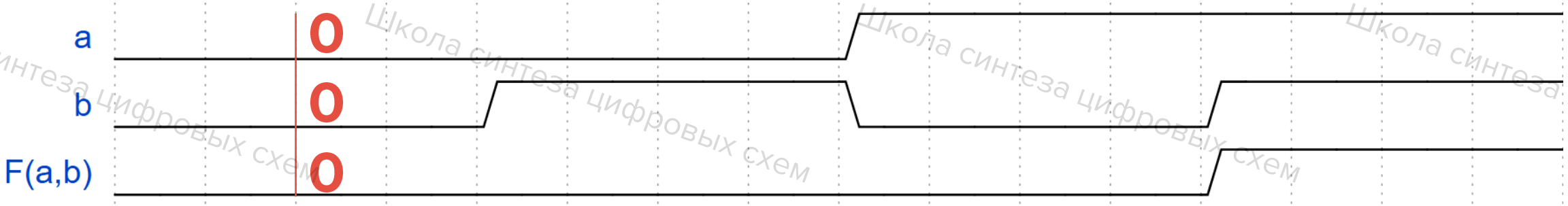
КАК ЧИТАТЬ ВРЕМЕННЫЕ ДИАГРАММЫ



a	b	F (a, b)
0	0	
0	1	
1	0	
1	1	

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

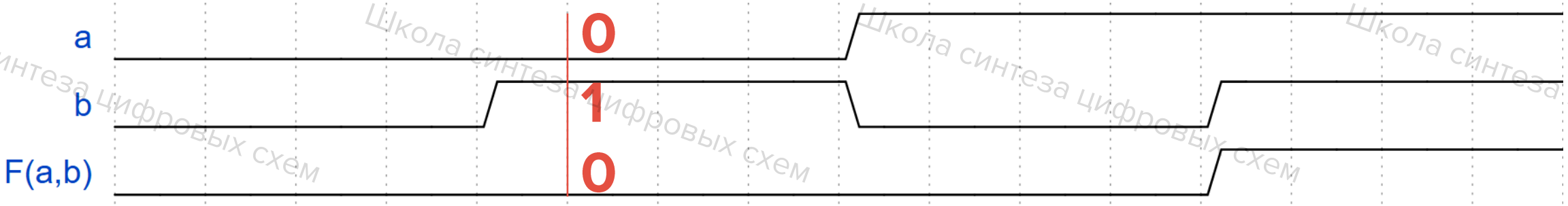
a = 0, b = 0



a	b	F (a, b)
0	0	0
0	1	
1	0	
1	1	

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

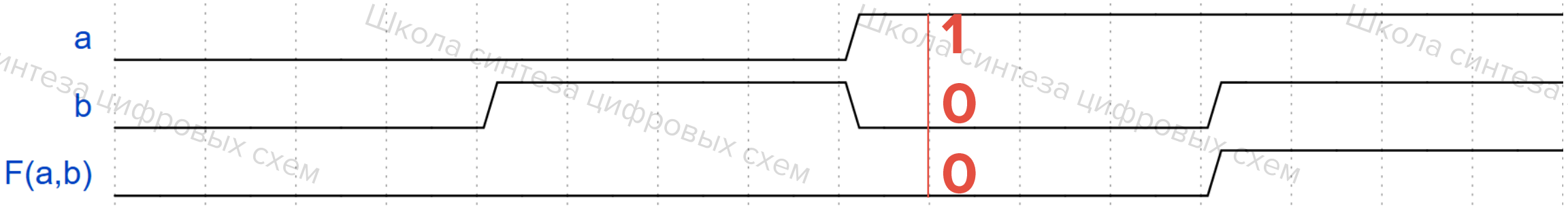
a = 0, b = 1



a	b	F (a, b)
0	0	0
0	1	0
1	0	
1	1	

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

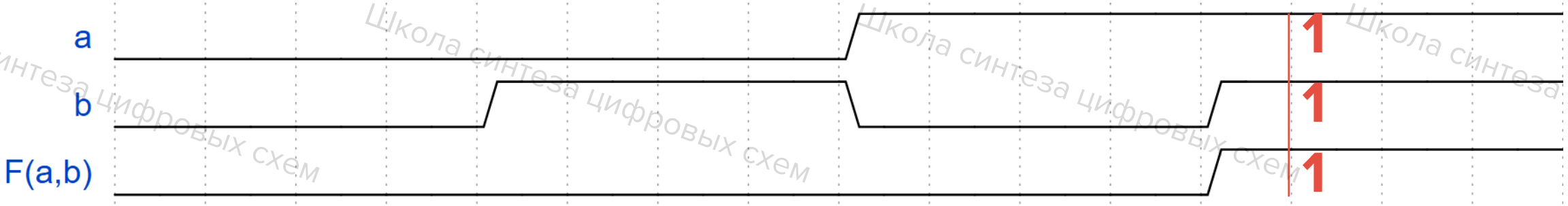
a = 1, b = 0



a	b	F (a, b)
0	0	0
0	1	0
1	0	0
1	1	

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

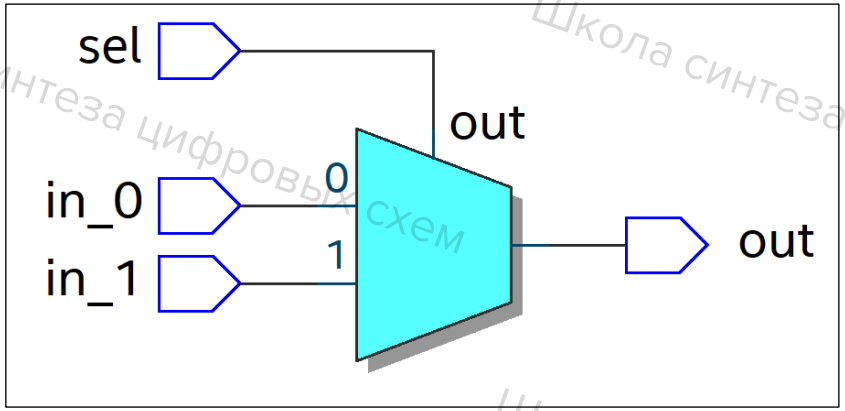
a = 1, b = 1



a	b	F (a, b)
0	0	0
0	1	0
1	0	0
1	1	1

МУЛЬТИПЛЕКСОР ИЗ 2 В 1 (MUX-1)

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ



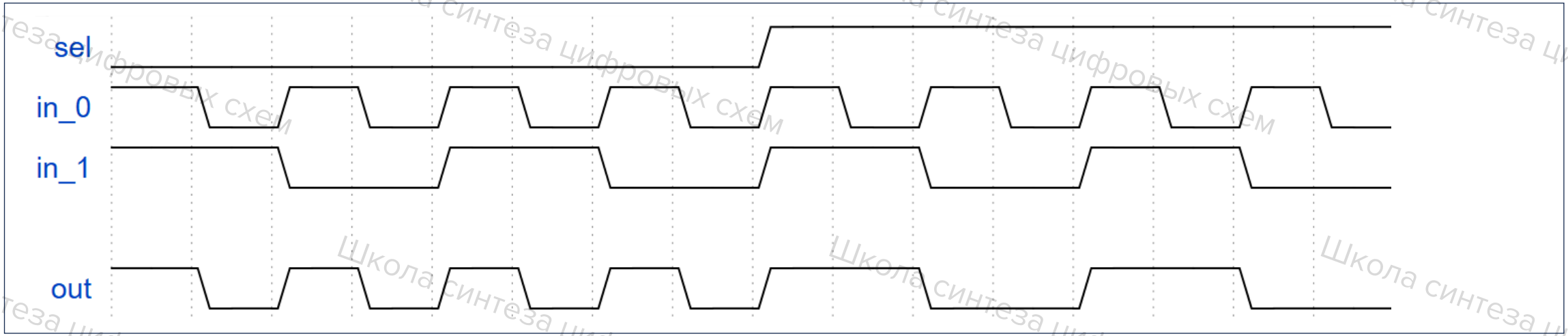
MUX-1

мультиплексор

из 2 в 1 – аппаратная реализация if/else

Если (sel = 1), то out = in_1

Если (sel = 0), то out = in_0



Школа синтеза цифровых схем

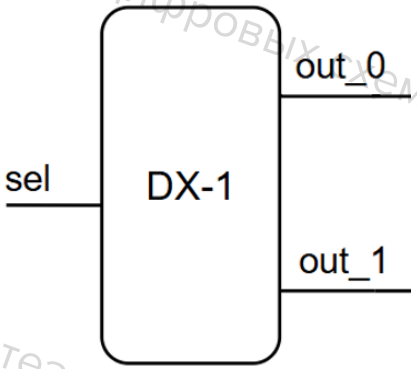
РЕАЛИЗАЦИЯ MUX-1 НА «И/ИЛИ/НЕ»

Школа синтеза цифровых схем

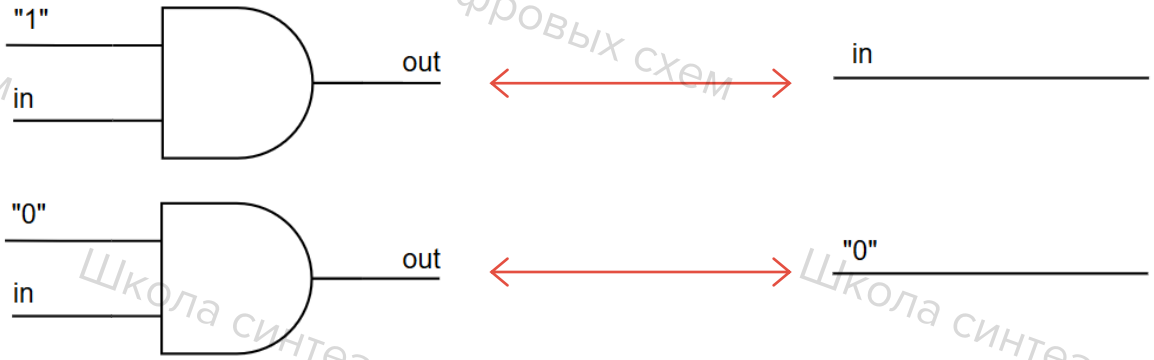
Для реализации MUX-1 мы воспользуемся системой **дешифратор + элементы «И»** в качестве ключей, выходы которых идут на элемент «ИЛИ»

«И» в качестве ключей — если на один вход элемента «И» подать «1», то он будет пропускать данные без изменений

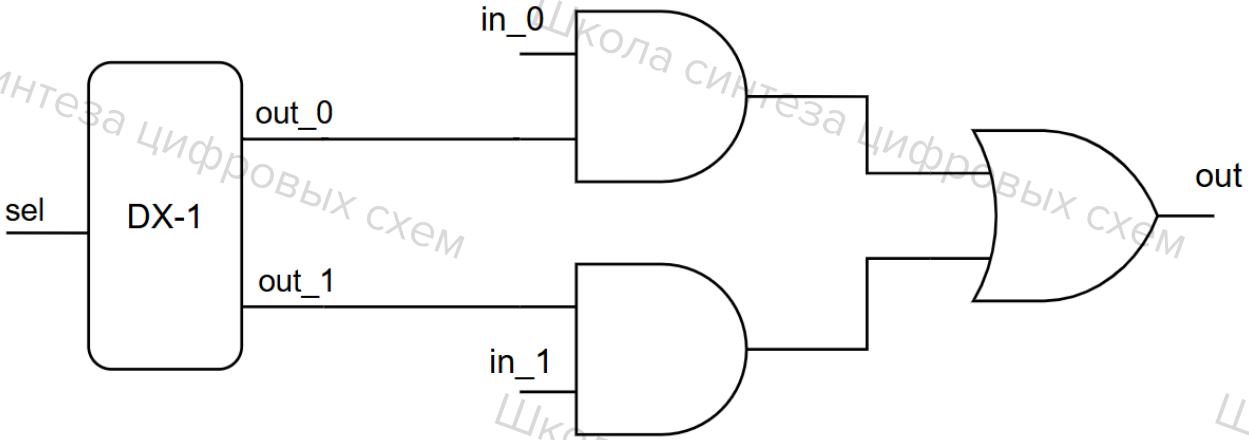
Дешифратор — модуль, который ставит «1» нужному элементу «И» в зависимости от сигнала **sel**

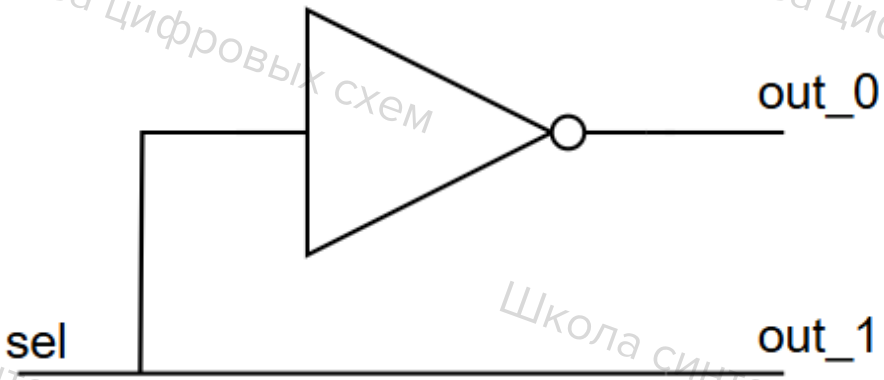


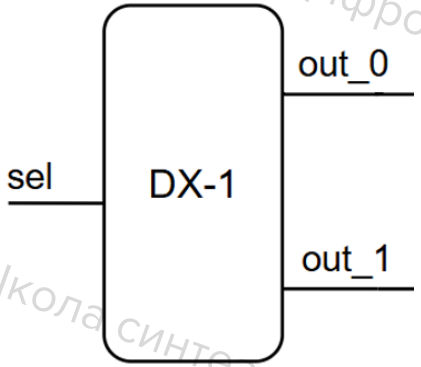
sel	out_0	out_1
0	1	0
1	0	1



КОНЦЕПТУАЛЬНАЯ СХЕМА MUX-1

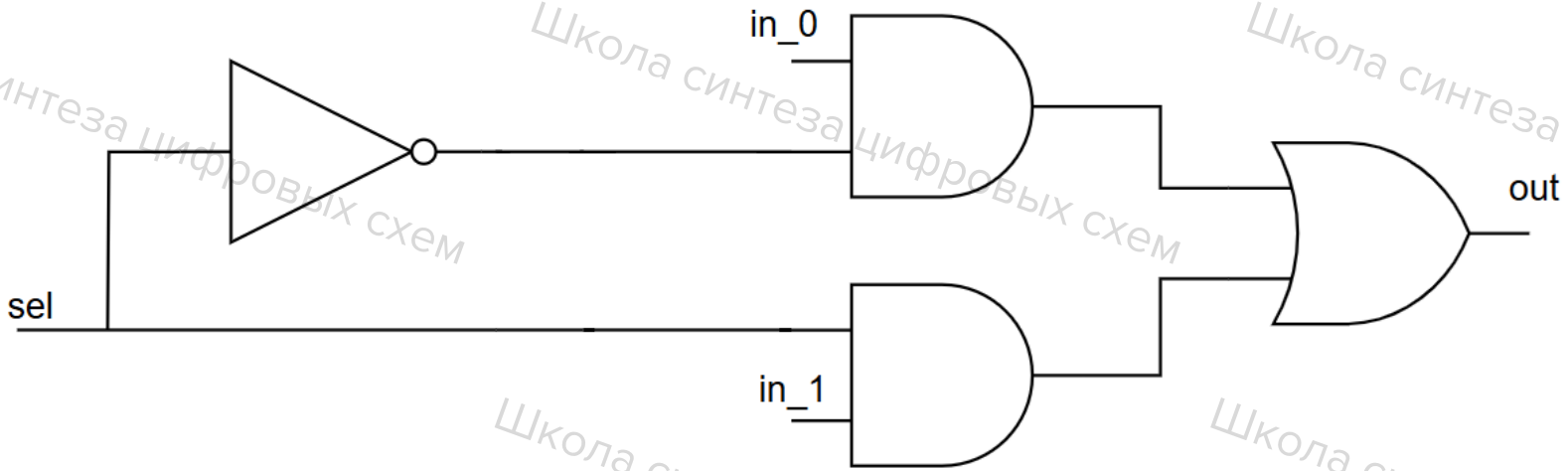


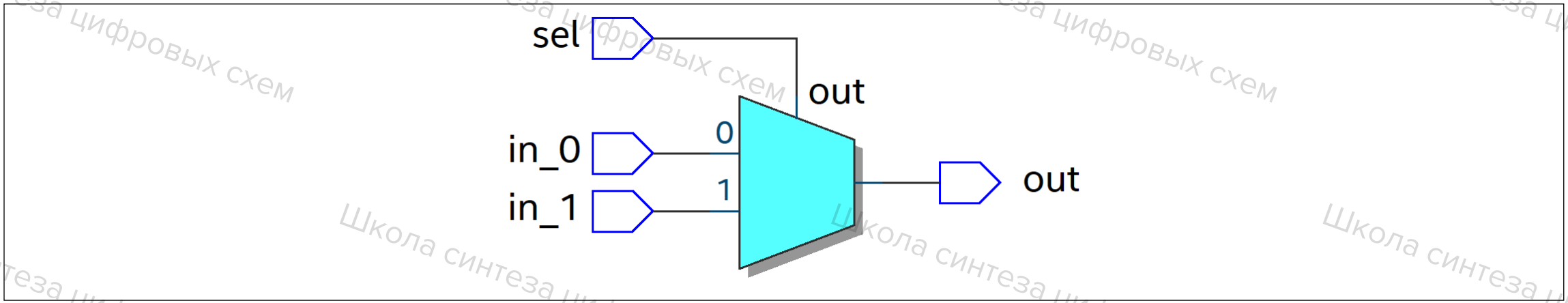




sel	out_0	out_1
0	1	0
1	0	1

МУЛЬТИПЛЕКСОР ИЗ 2 В 1 НА «И/ИЛИ/НЕ»





ПРЕВРАЩЕНИЕ SYSTEM VERILOG В NETLIST

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
module counter_3_bit (
        input               clk,
        input               rst,
        output logic [2:0] num
);

always_ff @(posedge clk) begin
        if (rst) num <= '0;
        else num <= num + 1;
end

endmodule
```

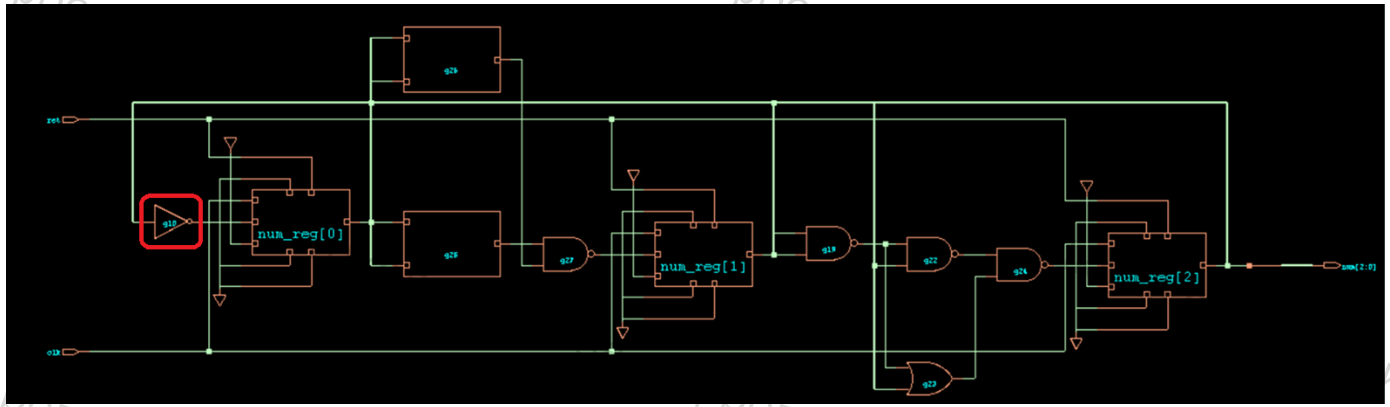


```
module counter_3_bit(clk, rst, num);
  input clk, rst;
  output [2:0] num;
  wire clk, rst;
  wire [2:0] num;
  wire n_6, n_7, n_8, n_27, n_28, n_29, n_30, n_31;
  not g10 (n_6, num[0]);
  nand g19 (n_27, num[0], num[1]);
  nand g22 (n_28, n_27, num[2]);
  or g23 (n_29, n_27, num[2]);
  nand g24 (n_8, n_28, n_29);
  or g25 (n_30, wc, num[0]);
  not gc (wc, num[1]);
  or g26 (n_31, num[1], wc0);
  not gc0 (wc0, num[0]);
  nand g27 (n_7, n_30, n_31);
  CDN_flop \num_reg[0] (.clk (clk), .d (n_6), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[0]));
  CDN_flop \num_reg[1] (.clk (clk), .d (n_7), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[1]));
  CDN_flop \num_reg[2] (.clk (clk), .d (n_8), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[2]));
endmodule

`ifdef RC_CDN_GENERIC_GATE
`else
module CDN_flop(clk, d, sena, aclr, apre, srl, srd, q);
  input clk, d, sena, aclr, apre, srl, srd;
  output q;
  wire clk, d, sena, aclr, apre, srl, srd;
  wire q;
  reg  qi;
  assign #1 q = qi;
  always
    @(posedge clk or posedge apre or posedge aclr)
      if (aclr)
        qi <= 0;
      else if (apre)
          qi <= 1;
        else if (srl)
            qi <= srd;
          else begin
            if (sena)
              qi <= d;
          end
  initial
    qi <= 1'b0;
endmodule
`endif
```

NOT В NETLIST И НА СХЕМЕ

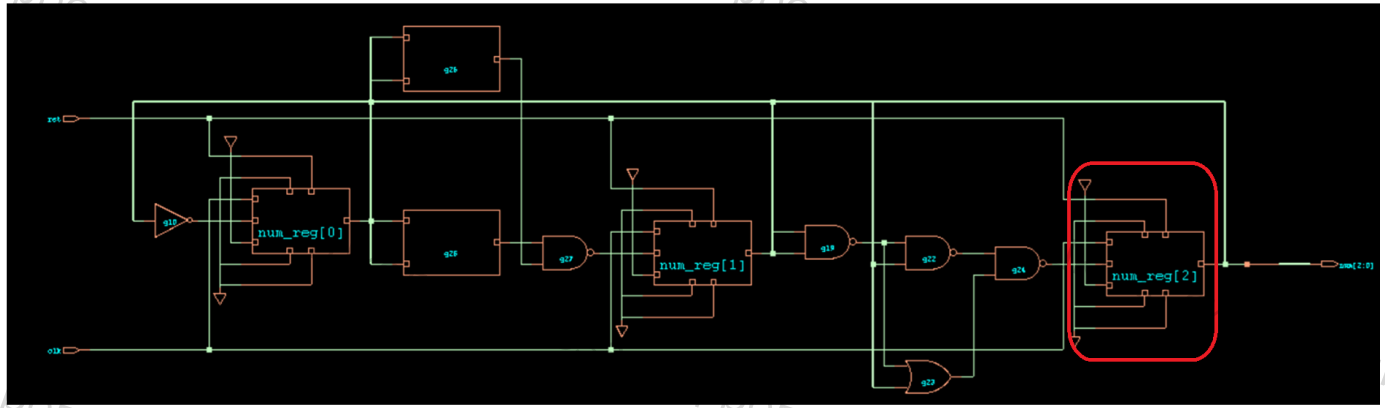
```
module counter_3_bit(clk, rst, num);
  input clk, rst;
  output [2:0] num;
  wire clk, rst;
  wire [2:0] num;
  wire n_6, n_7, n_8, n_27, n_28, n_29, n_30, n_31;
  not g10 (n_6, num[0]);
  nand g19 (n_27, num[0], num[1]);
  nand g22 (n_28, n_27, num[2]);
  or g23 (n_29, n_27, num[2]);
  nand g24 (n_8, n_28, n_29);
  or g25 (n_30, wc, num[0]);
  not gc (wc, num[1]);
  or g26 (n_31, num[1], wc0);
  not gc0 (wc0, num[0]);
  nand g27 (n_7, n_30, n_31);
  CDN_flop \num_reg[0] (.clk (clk), .d (n_6), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[0]));
  CDN_flop \num_reg[1] (.clk (clk), .d (n_7), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[1]));
  CDN_flop \num_reg[2] (.clk (clk), .d (n_8), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[2]));
endmodule
```



2 БИТ СЧЁТЧИКА В NETLIST И НА СХЕМЕ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
module counter_3_bit(clk, rst, num);
  input clk, rst;
  output [2:0] num;
  wire clk, rst;
  wire [2:0] num;
  wire n_6, n_7, n_8, n_27, n_28, n_29, n_30, n_31;
  not g10 (n_6, num[0]);
  nand g19 (n_27, num[0], num[1]);
  nand g22 (n_28, n_27, num[2]);
  or g23 (n_29, n_27, num[2]);
  nand g24 (n_8, n_28, n_29);
  or g25 (n_30, wc, num[0]);
  not gc (wc, num[1]);
  or g26 (n_31, num[1], wc0);
  not gc0 (wc0, num[0]);
  nand g27 (n_7, n_30, n_31);
  CDN_flop \num_reg[0] (.clk (clk), .d (n_6), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[0]));
  CDN_flop \num_reg[1] (.clk (clk), .d (n_7), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[1]));
  CDN_flop \num_reg[2] (.clk (clk), .d (n_8), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[2]));
endmodule
```



ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

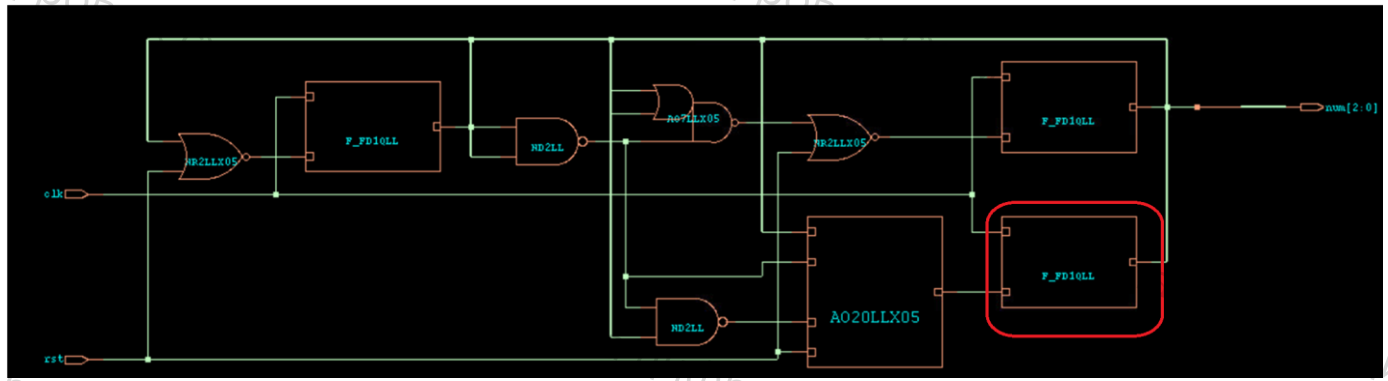
ЗАМЕНА ЭЛЕМЕНТОВ НА ЯЧЕЙКИ ИЗ ТЕХЛИБЫ

```
module counter_3_bit(clk, rst, num);
  input clk, rst;
  output [2:0] num;
  wire clk, rst;
  wire [2:0] num;
  wire n_6, n_7, n_8, n_27, n_28, n_29, n_30, n_31;
  not g10 (n_6, num[0]);
  nand g19 (n_27, num[0], num[1]);
  nand g22 (n_28, n_27, num[2]);
  or g23 (n_29, n_27, num[2]);
  nand g24 (n_8, n_28, n_29);
  or g25 (n_30, wc, num[0]);
  not gc (wc, num[1]);
  or g26 (n_31, num[1], wc0);
  not gc0 (wc0, num[0]);
  nand g27 (n_7, n_30, n_31);
  CDN_flop \num_reg[0] (.clk (clk), .d (n_6), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[0]));
  CDN_flop \num_reg[1] (.clk (clk), .d (n_7), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[1]));
  CDN_flop \num_reg[2] (.clk (clk), .d (n_8), .sena (1'b1), .aclr
       (1'b0), .apre (1'b0), .srl (rst), .srd (1'b0), .q (num[2]));
endmodule
`ifdef RC_CDN_GENERIC_GATE
`else
module CDN_flop(clk, d, sena, aclr, apre, srl, srd, q);
  input clk, d, sena, aclr, apre, srl, srd;
  output q;
  wire clk, d, sena, aclr, apre, srl, srd;
  wire q;
  reg  qi;
  assign #1 q = qi;
  always
    @(posedge clk or posedge apre or posedge aclr)
      if (aclr)
        qi <= 0;
      else if (apre)
          qi <= 1;
        else if (srl)
            qi <= srd;
          else begin
            if (sena)
              qi <= d;
          end
  initial
    qi <= 1'b0;
endmodule
`endif
```

```
module counter_3_bit(clk, rst, num);
  input clk, rst;
  output [2:0] num;
  wire clk, rst;
  wire [2:0] num;
  wire n_0, n_1, n_2, n_3, n_4, n_5;
  F_FD1QLL \num_reg[2] (.CP (clk), .D (n_5), .Q (num[2]));
  F_FD1QLL \num_reg[1] (.CP (clk), .D (n_4), .Q (num[1]));
  AO20LLX05 g57__2398(.A (num[2]), .B (n_2), .C (n_1), .D (rst), .Z
       (n_5));
  NR2LLX05 g60__5107(.A (n_3), .B (rst), .Z (n_4));
  F_FD1QLL \num_reg[0] (.CP (clk), .D (n_0), .Q (num[0]));
  AO7LLX05 g62__6260(.A (num[1]), .B (num[0]), .C (n_2), .Z (n_3));
  ND2LL g59__4319(.A (n_2), .B (num[2]), .Z (n_1));
  ND2LL g64__8428(.A (num[0]), .B (num[1]), .Z (n_2));
  NR2LLX05 g63__5526(.A (num[0]), .B (rst), .Z (n_0));
endmodule
```

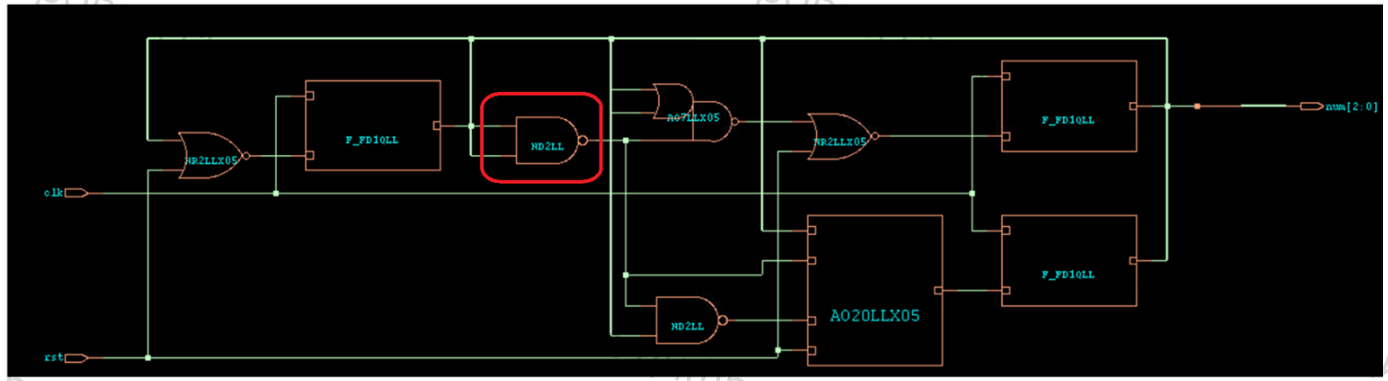
2 БИТ СЧЁТЧИКА В NETLISTИ НА СХЕМЕ

```
 7 module counter_3_bit(clk, rst, num);
 8   input clk, rst;
 9   output [2:0] num;
10   wire clk, rst;
11   wire [2:0] num;
12   wire n_0, n_1, n_2, n_3, n_4, n_5;
13   F_FD1QLL \num_reg[2] (.CP (clk), .D (n_5), .Q (num[2]));
14   F_FD1QLL \num_reg[1] (.CP (clk), .D (n_4), .Q (num[1]));
15   AO20LLX05 g57__2398(.A (num[2]), .B (n_2), .C (n_1), .D (rst), .Z
16        (n_5));
17   NR2LLX05 g60__5107(.A (n_3), .B (rst), .Z (n_4));
18   F_FD1QLL \num_reg[0] (.CP (clk), .D (n_0), .Q (num[0]));
19   AO7LLX05 g62__6260(.A (num[1]), .B (num[0]), .C (n_2), .Z (n_3));
20   ND2LL g59__4319(.A (n_2), .B (num[2]), .Z (n_1));
21   ND2LL g64__8428(.A (num[0]), .B (num[1]), .Z (n_2));
22   NR2LLX05 g63__5526(.A (num[0]), .B (rst), .Z (n_0));
23 endmodule
```



NAND В NETLIST И НА СХЕМЕ

```
 7 module counter_3_bit(clk, rst, num);
 8   input clk, rst;
 9   output [2:0] num;
10   wire clk, rst;
11   wire [2:0] num;
12   wire n_0, n_1, n_2, n_3, n_4, n_5;
13   F_FD1QLL \num_reg[2] (.CP (clk), .D (n_5), .Q (num[2]));
14   F_FD1QLL \num_reg[1] (.CP (clk), .D (n_4), .Q (num[1]));
15   AO20LLX05 g57__2398(.A (num[2]), .B (n_2), .C (n_1), .D (rst), .Z
16        (n_5));
17   NR2LLX05 g60__5107(.A (n_3), .B (rst), .Z (n_4));
18   F_FD1QLL \num_reg[0] (.CP (clk), .D (n_0), .Q (num[0]));
19   AO7LLX05 g62__6260(.A (num[1]), .B (num[0]), .C (n_2), .Z (n_3));
20   ND2LL g59__4319(.A (n_2), .B (num[2]), .Z (n_1));
21   ND2LL g64__8428(.A (num[0]), .B (num[1]), .Z (n_2));
22   NR2LLX05 g63__5526(.A (num[0]), .B (rst), .Z (n_0));
23 endmodule
```



ФИЗИЧЕСКОЕ ПРОЕКТИРОВАНИЕ

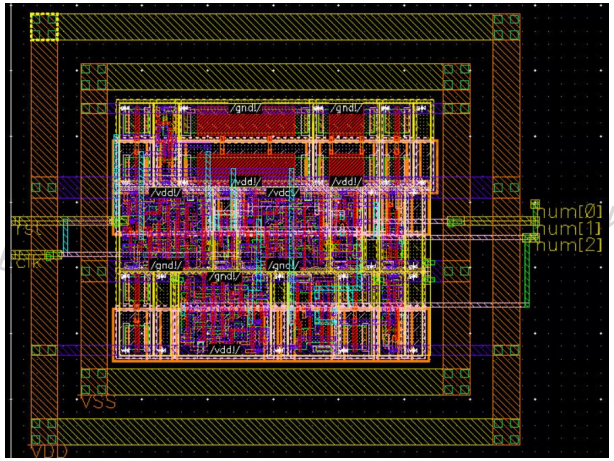
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

Цифровой тополог переводит **netlist** в топологию устройства **.GDS/GDSII файл**, которую уже можно будет отправить на производство

```
module counter_3_bit(clk, rst, num);
  input clk, rst;
  output [2:0] num;
  wire clk, rst;
  wire [2:0] num;
  wire n_0, n_1, n_2, n_3, n_4, n_5;
  F_FD1QLL \num_reg[2] (.CP (clk), .D (n_5), .Q (num[2]));
  F_FD1QLL \num_reg[1] (.CP (clk), .D (n_4), .Q (num[1]));
  AO20LLX05 g57__2398(.A (num[2]), .B (n_2), .C (n_1), .D (rst), .Z
       (n_5));
  NR2LLX05 g60__5107(.A (n_3), .B (rst), .Z (n_4));
  F_FD1QLL \num_reg[0] (.CP (clk), .D (n_0), .Q (num[0]));
  AO7LLX05 g62__6260(.A (num[1]), .B (num[0]), .C (n_2), .Z (n_3));
  ND2LL g59__4319(.A (n_2), .B (num[2]), .Z (n_1));
  ND2LL g64__8428(.A (num[0]), .B (num[1]), .Z (n_2));
  NR2LLX05 g63__5526(.A (num[0]), .B (rst), .Z (n_0));
endmodule
```

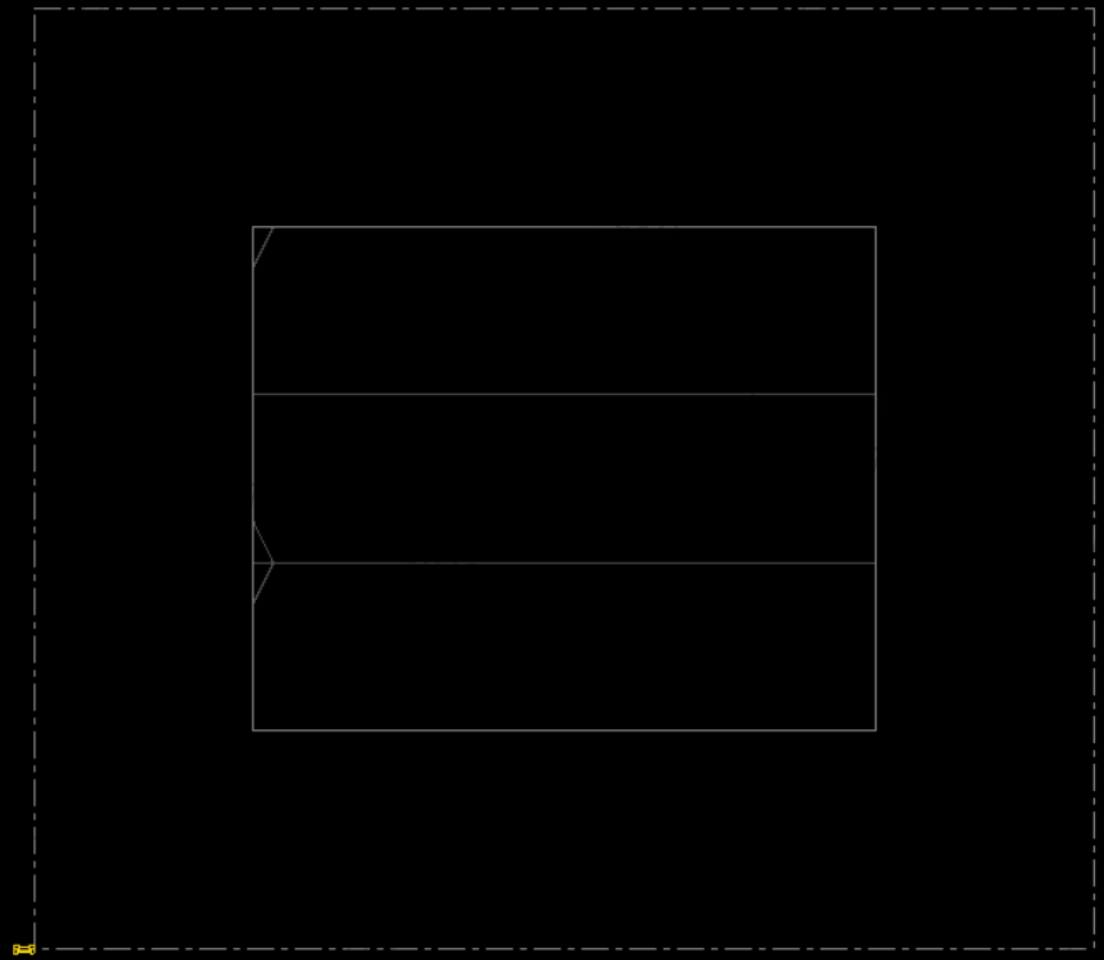
+

Технологическая библиотека



ПОДГОТОВКА ТОПОЛОГИИ. DIE BOUNDARY

В Die Boundary располагаются
входные/выходные пины
и кольца питания



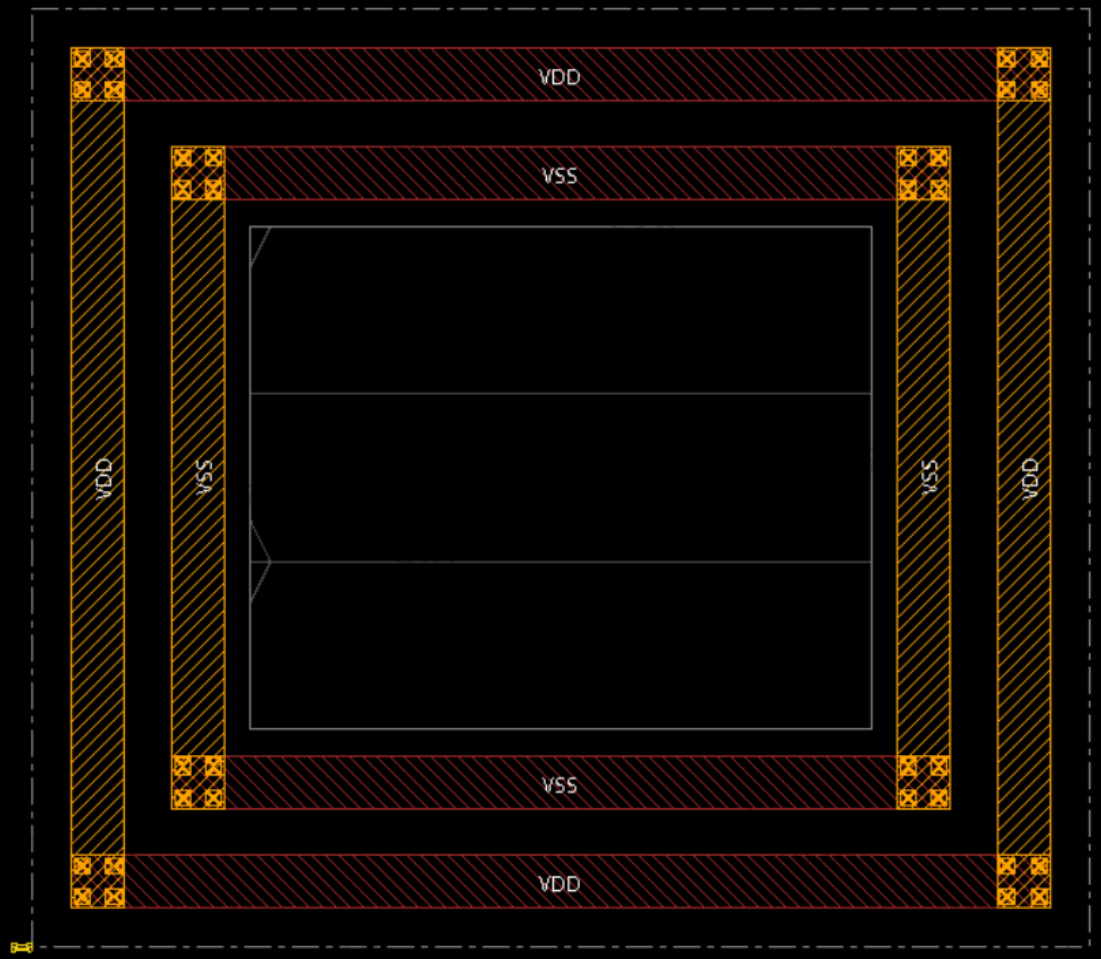
КОЛЬЦА ПИТАНИЯ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

Кольца питания нужны для подведения питания (VDD) и земли (VSS) от внешних источников

Делаются обычно в верхних металлах

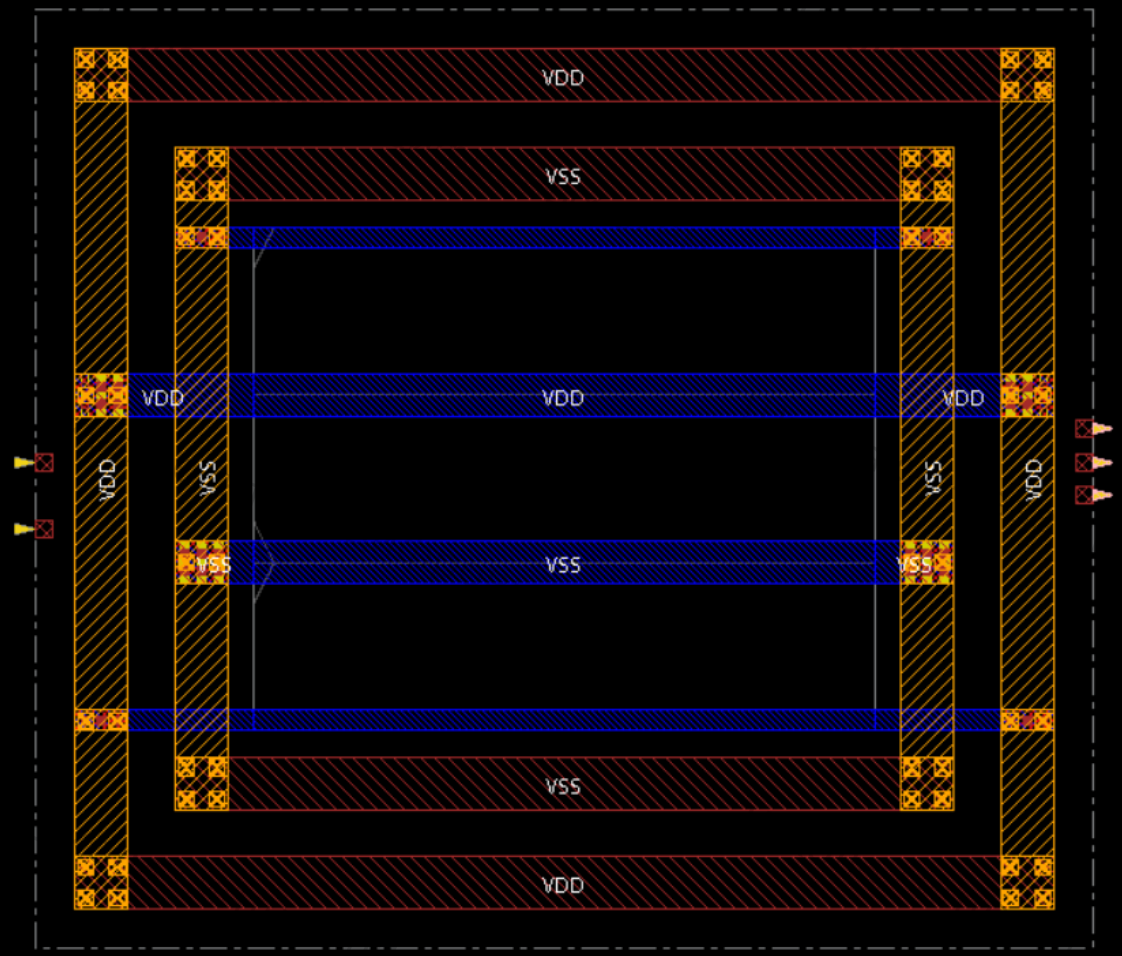
- Металл-5
- Металл-6



ЛИНИИ ПИТАНИЯ ДЛЯ ЯЧЕЕК И ПИНЫ

школа синтеза цифровых схем

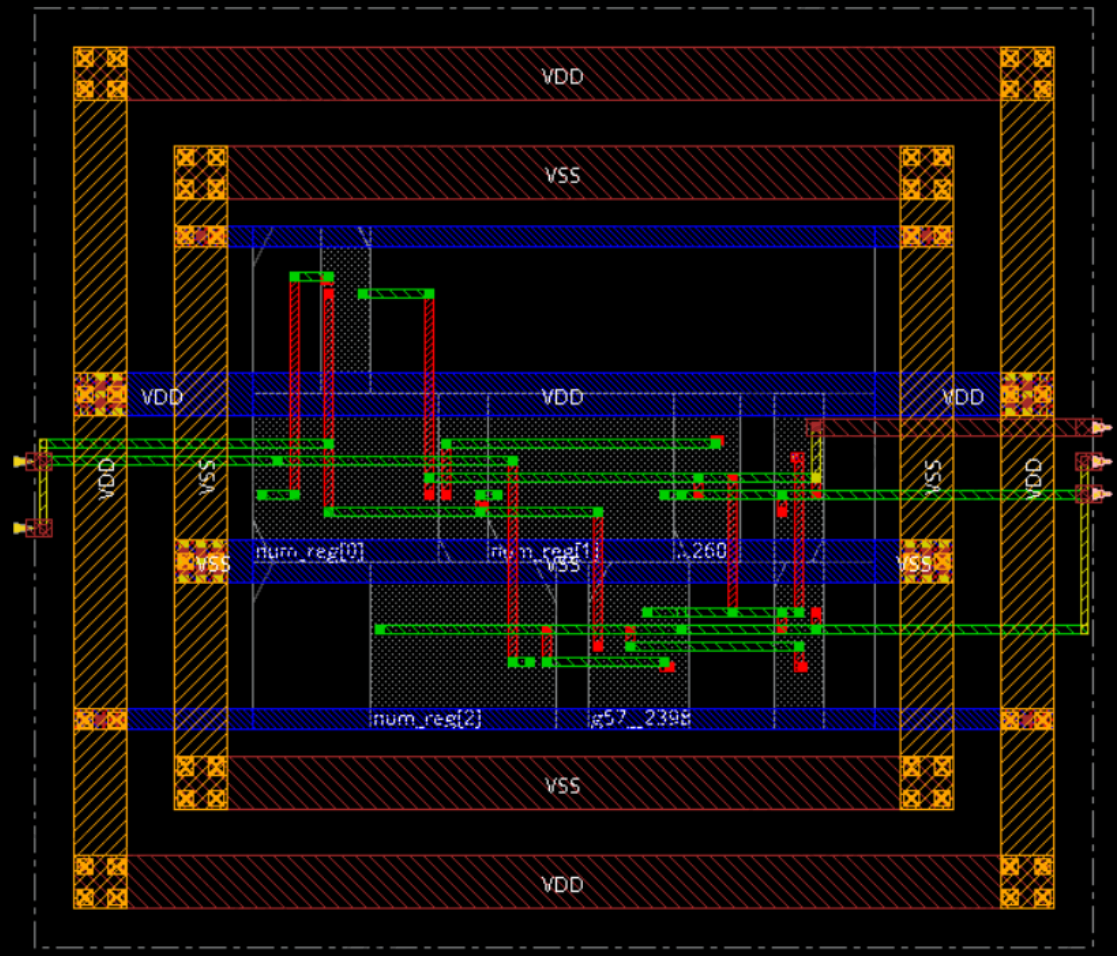
Линии питания для ячеек делаются в первом металле, располагаются автоматически



- Металл-6

РАСПОЛОЖЕНИЕ ТОПОЛОГИИ (PLACE_DESIGN)

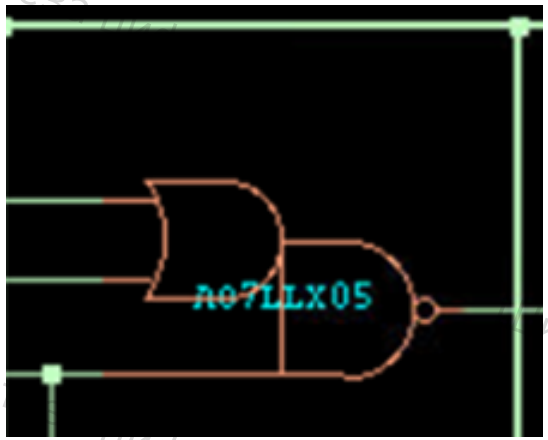
Автоматически располагаются
и соединяются ячейки,
соединения неоптимизированные

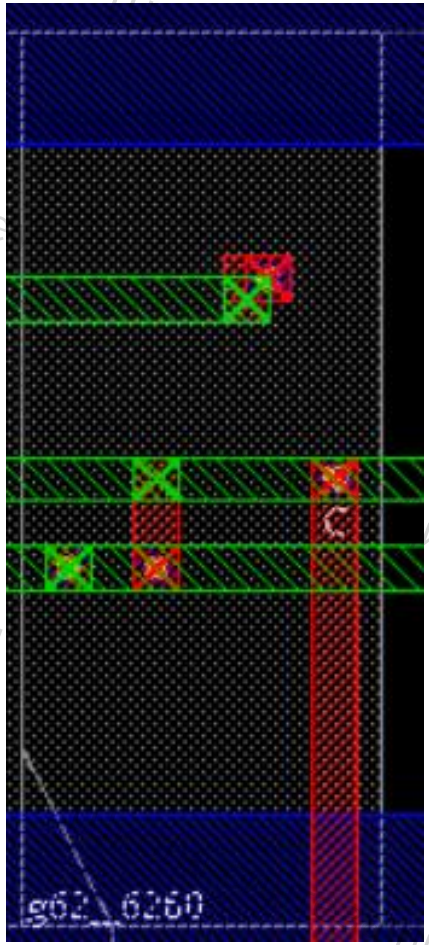


ЯЧЕЙКА В NETLIST И НА ТОПОЛОГИИ

школа синтеза цифровых схем

```
module counter_3_bit(clk, rst, num);
  input clk, rst;
  output [2:0] num;
  wire clk, rst;
  wire [2:0] num;
  wire n_0, n_1, n_2, n_3, n_4, n_5;
  F_FD1QLL \num_reg[2] (.CP (clk), .D (n_5), .Q (num[2]));
  F_FD1QLL \num_reg[1] (.CP (clk), .D (n_4), .Q (num[1]));
  AO20LLX05 g57__2398(.A (num[2]), .B (n_2), .C (n_1), .D (rst), .Z
       (n_5));
  NR2LLX05 g60__5107(.A (n_3), .B (rst), .Z (n_4));
  F_FD1QLL \num_reg[0] (.CP (clk), .D (n_0), .Q (num[0]));
  AO7LLX05 g62__6260(.A (num[1]), .B (num[0]), .C (n_2), .Z (n_3));
  ND2LL g59__4319(.A (n_2), .B (num[2]), .Z (n_1));
  ND2LL g64__8428(.A (num[0]), .B (num[1]), .Z (n_2));
  NR2LLX05 g63__5526(.A (num[0]), .B (rst), .Z (n_0));
endmodule
```



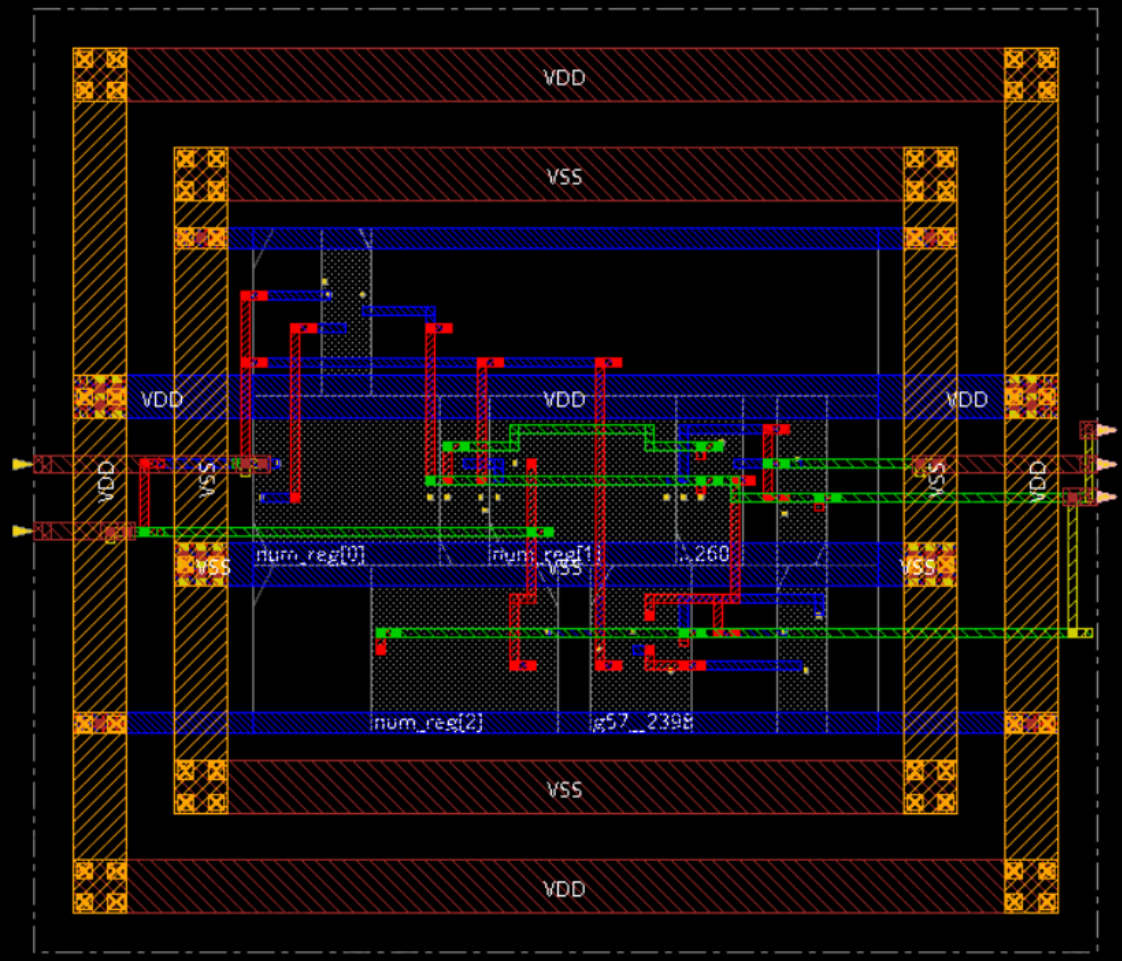


ОПТИМИЗАЦИЯ ДОРОЖЕК (ROUTE DESIGN)

школа синтеза цифровых схем

Route design

Оптимизирует дорожки
и убирает DRC ошибки

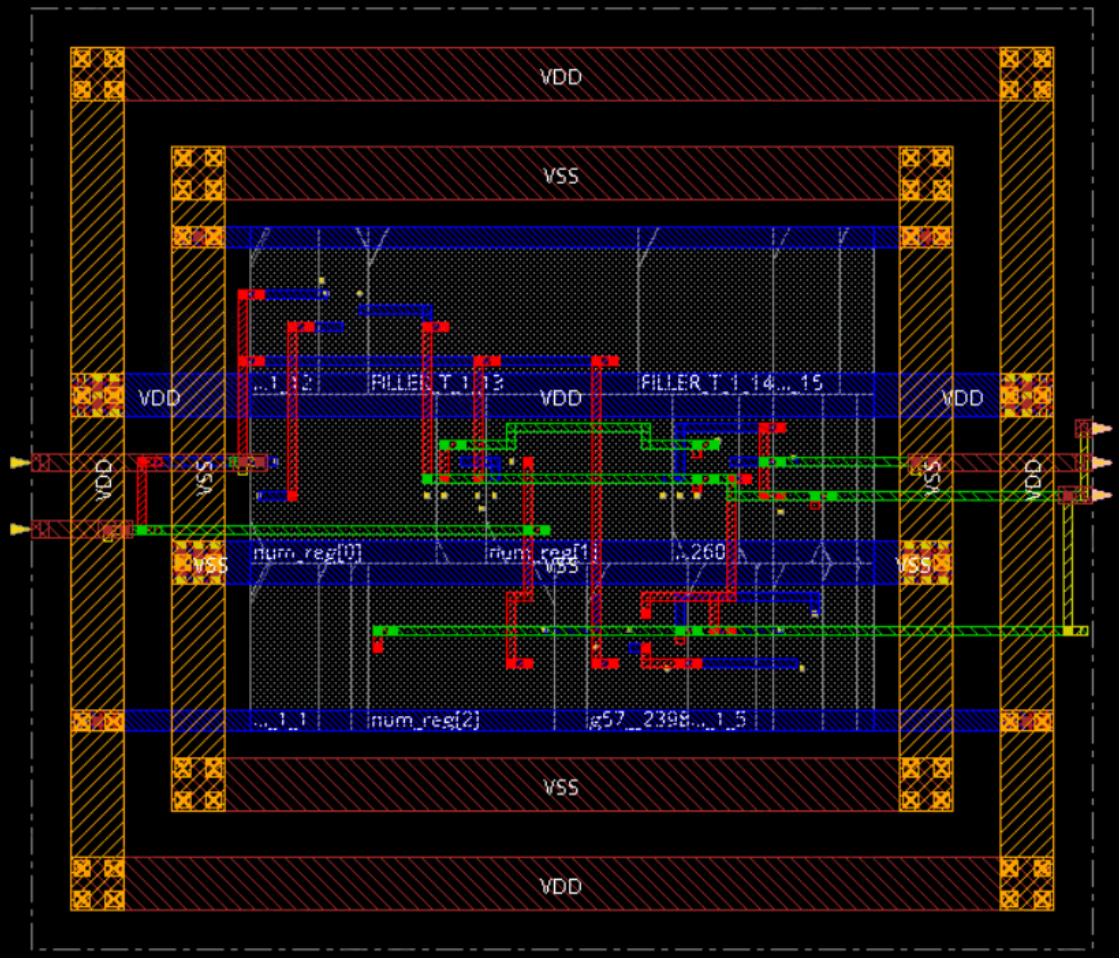


FILLER CELLS (ПУСТЫЕ ЯЧЕЙКИ)

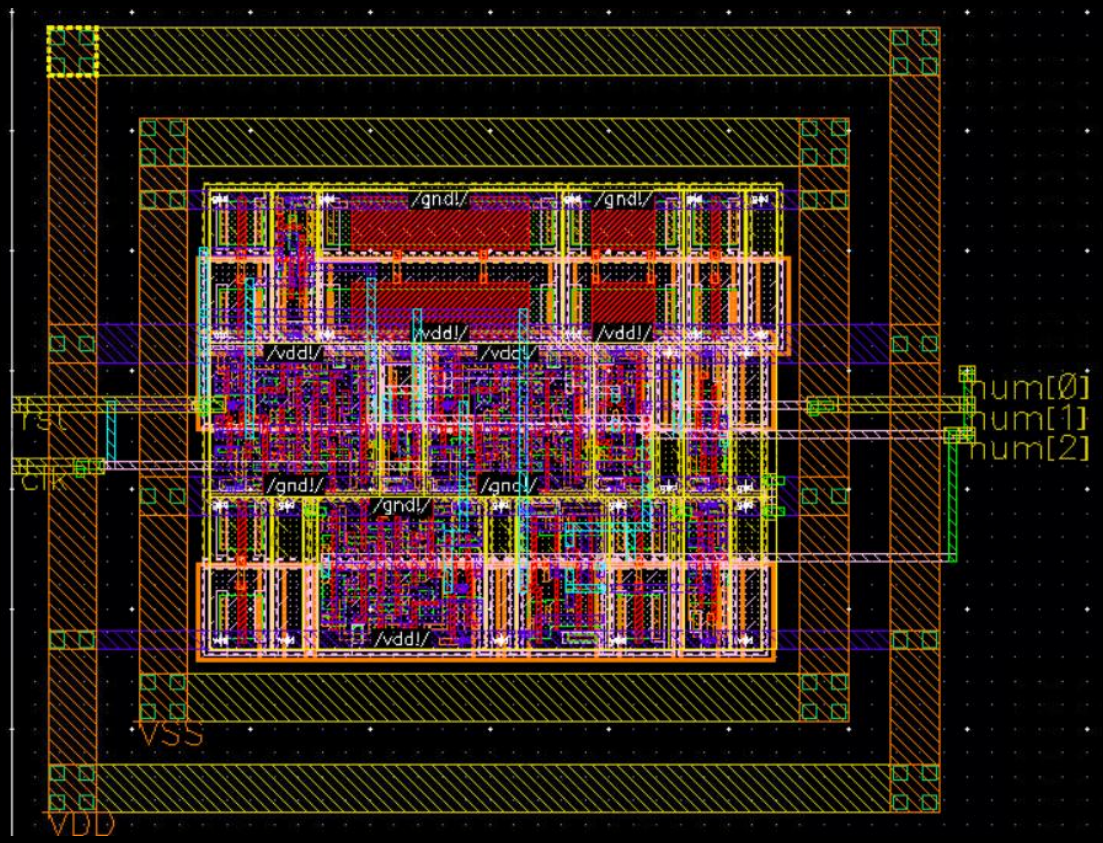
Filler Cells

филлеры, пустые ячейки

Нужны для того, чтобы убрать неприятные технологические эффекты, связанные, например, с различным тепловым расширением легированного и нелегированного Si от температуры



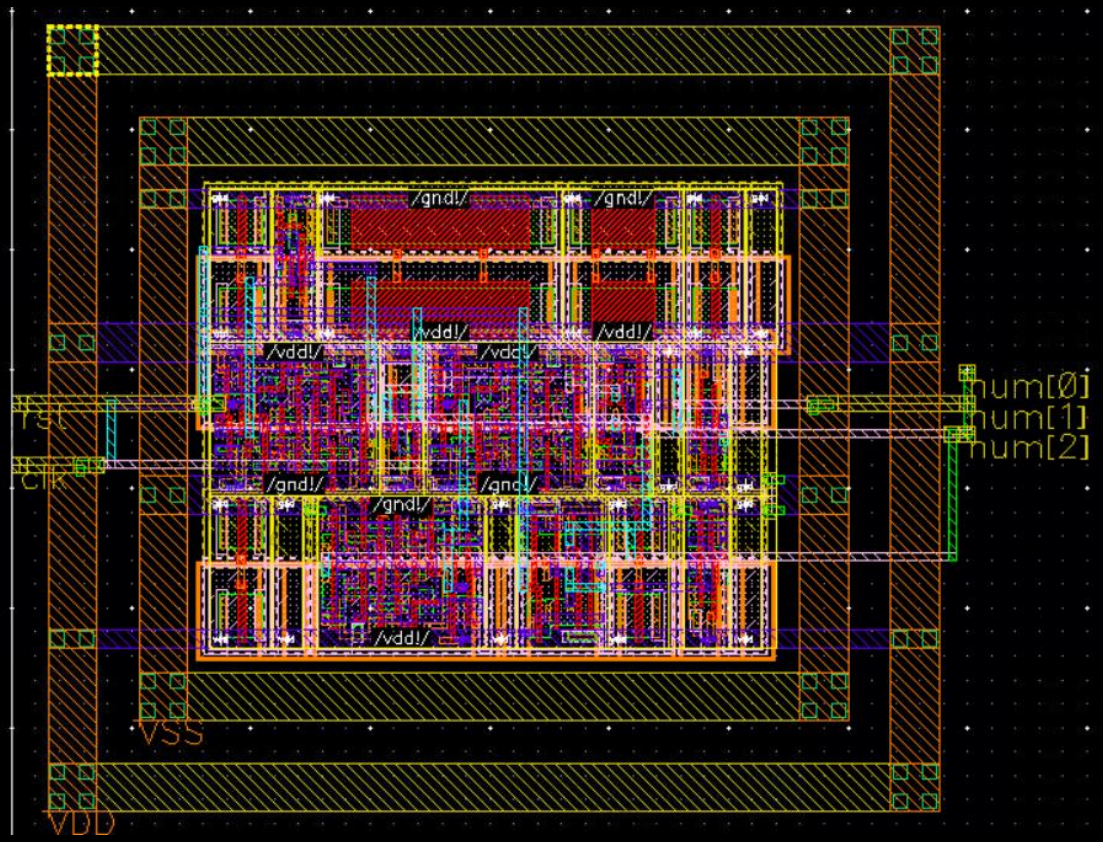
ЭКСТРАКЦИЯ НИЖНИХ СЛОЕВ (CADENCE VIRTUOSO)

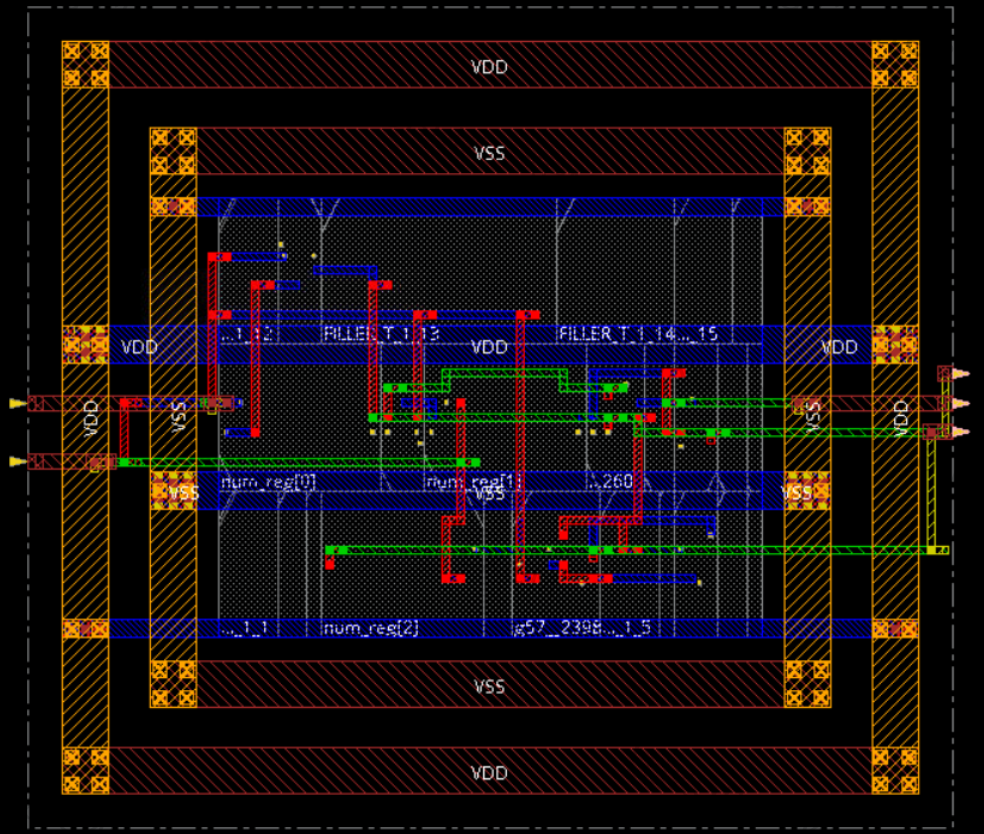


Нижние слои

n+, p+, n-карман, поликремний и тд — технологические слои, находящиеся под металлизацией

НИЖНИЕ И ВЕРХНИЕ СЛОИ





FPGA

школа синтеза цифровых схем

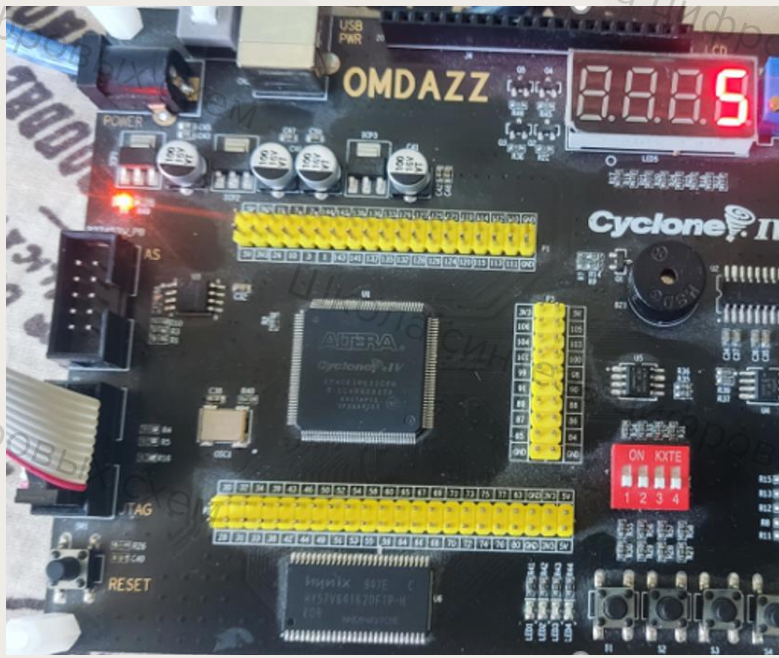
Field Programming Gate Array

Микросхемы программируемой логики

Программируемой — значит, изменяемой при помощи записи в память

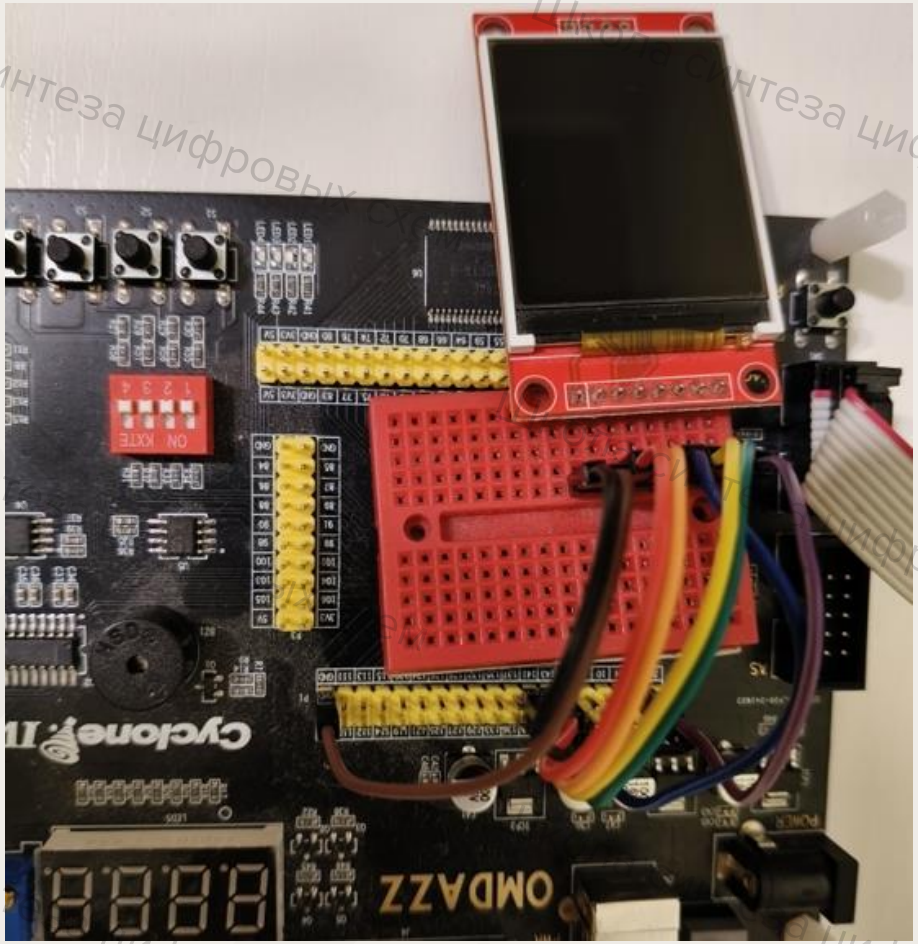
В основе технологии лежит пара основных приёмов:

- **01 | LUT**
  Универсальный комбинационный компонент **Look-Up Table — таблица истинности**
  Прошивая ячейки памяти, можно получить любую комбинационную схему
- **02 | Программируемые соединения**
  Позволяют собрать схему из имеющихся внутри FPGA компонентов



ПРОШИВКА ДИСПЛЕЯ С ПОМОЩЬЮ FPGA

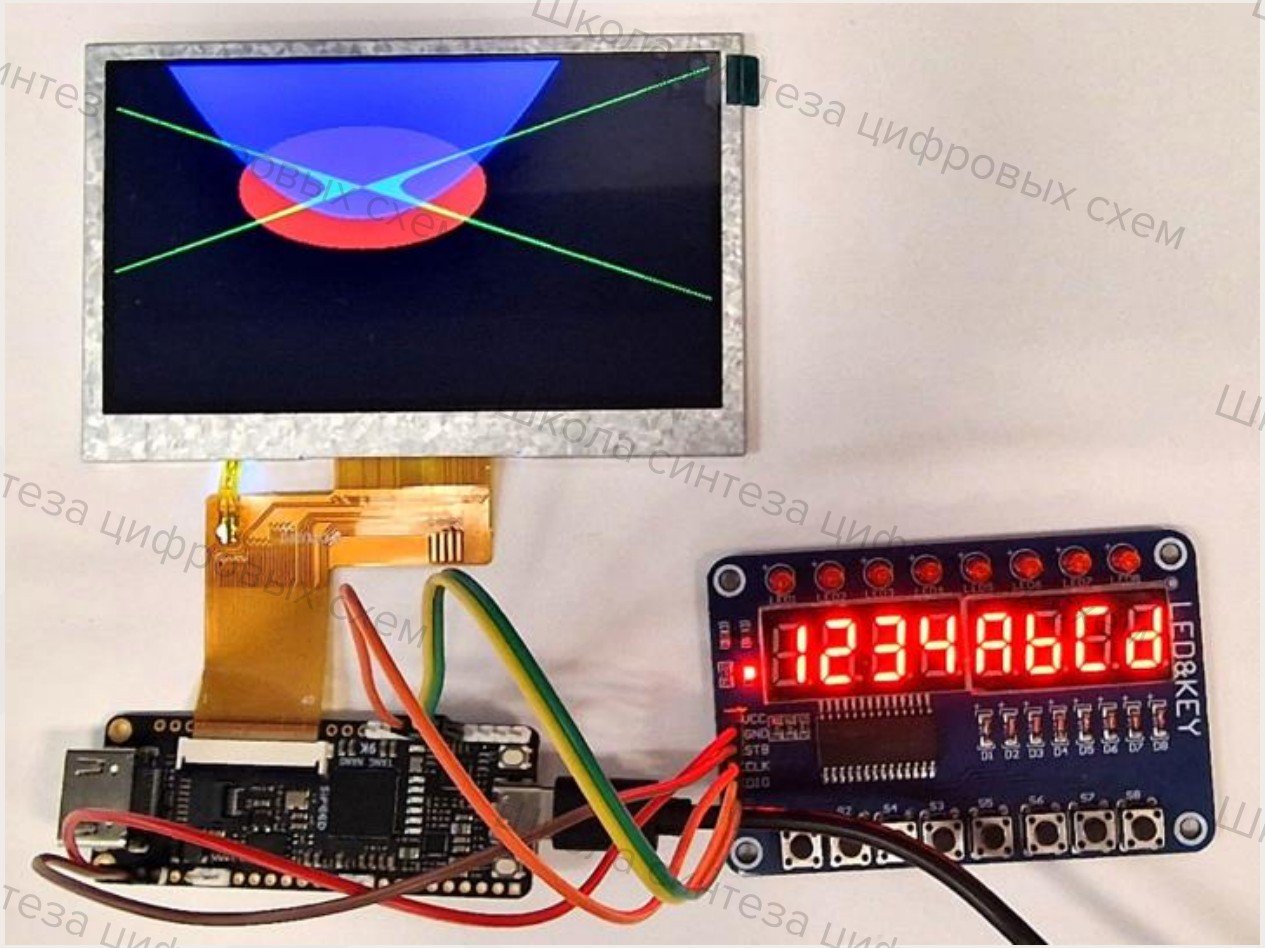
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ





ФИГУРЫ НА ЭКРАНЕ С ПОМОЩЬЮ FPGA

школа синтеза цифровых схем



Школа синтеза цифровых схем

ПРИМЕРЫ ИНДУСТРИАЛЬНОГО ПРИМЕНЕНИЯ FPGA

FPGA используется во многих потребительских продуктах:



Автомобили



Медицинское оборудование



Устройства обработки медиа-информации

01 | Пример:

В автомобилях **Mercedes Benz** S-класса в системах навигации, круиз контроля, звуковоспроизведения используется **более 10 FPGA** фирмы Xilinx

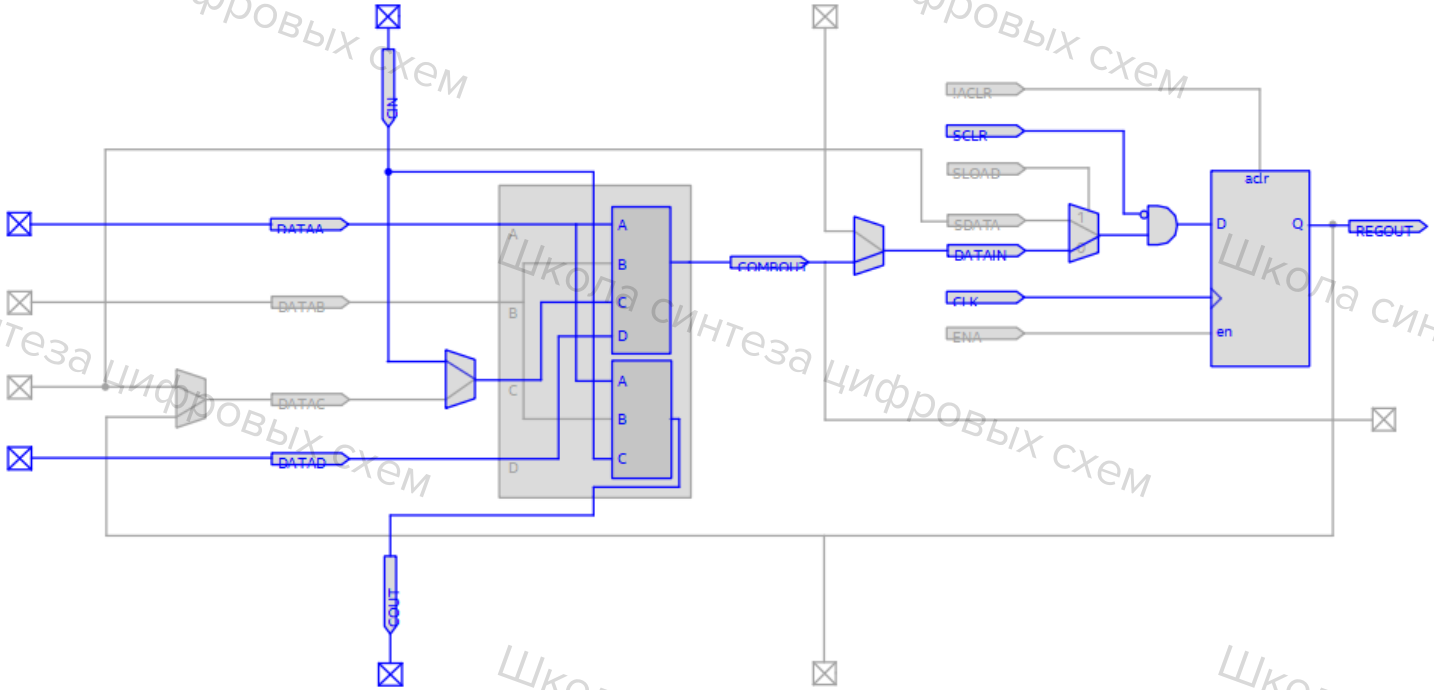
02 | Пример:

Илон Маск использует FPGA (Versal AI Core от AMD) в спутниках Starlink

Для обработки сигналов и формирования луча

школа синтеза цифровых схем

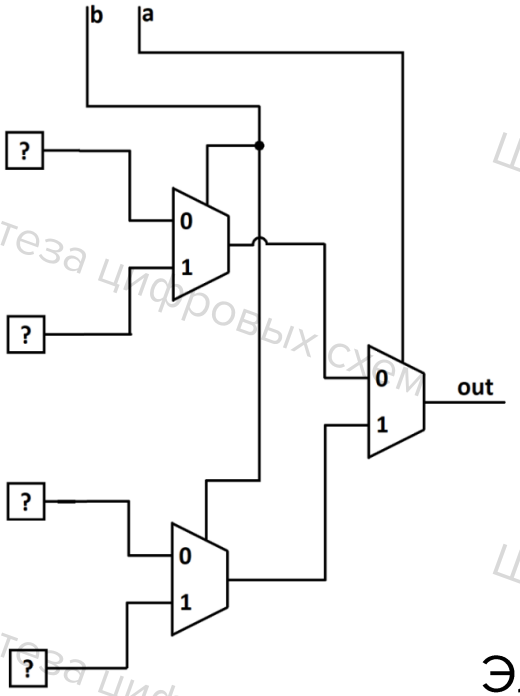
УНИВЕРСАЛЬНАЯ ЯЧЕЙКА FPGA И ПРОГРАММАТОР



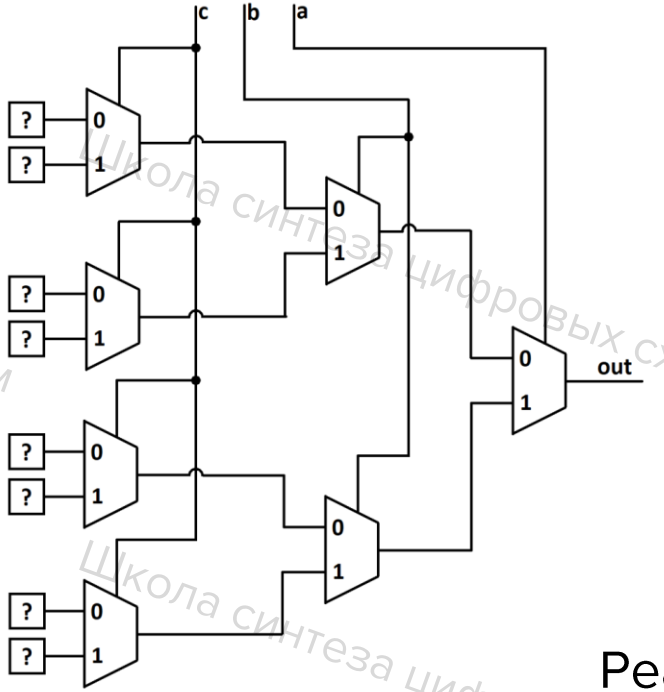


LUT (LOOK-UP TABLE)

Программатор присваивает ячейкам памяти «0» и «1» (вместо знаков вопроса) согласно таблице истинности, и данная конструкция начинает работать как комбинационная функция, которой LUT был прошит

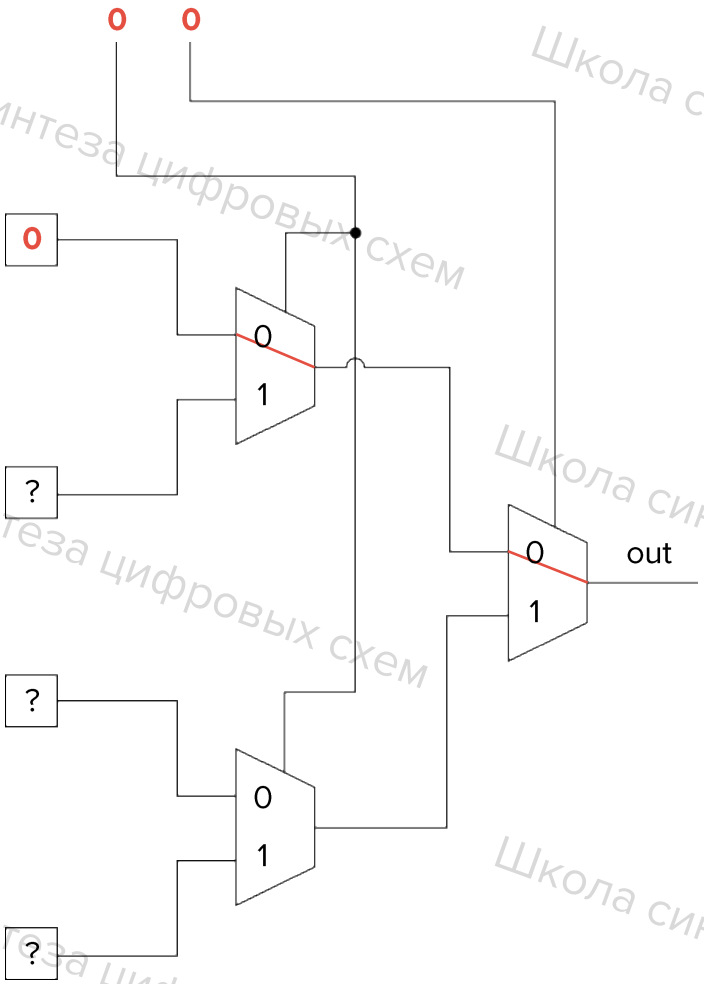


Элемент «И»



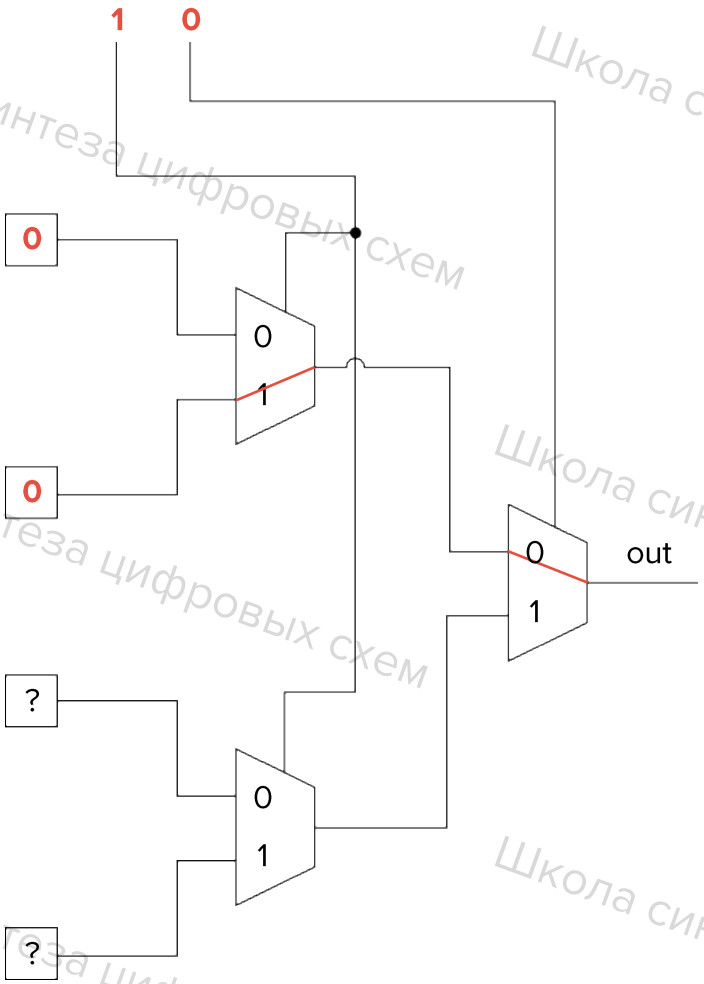
Реализация «A ^ B ^ C»

ЭЛЕМЕНТ «И» ЧЕРЕЗ LUT-2



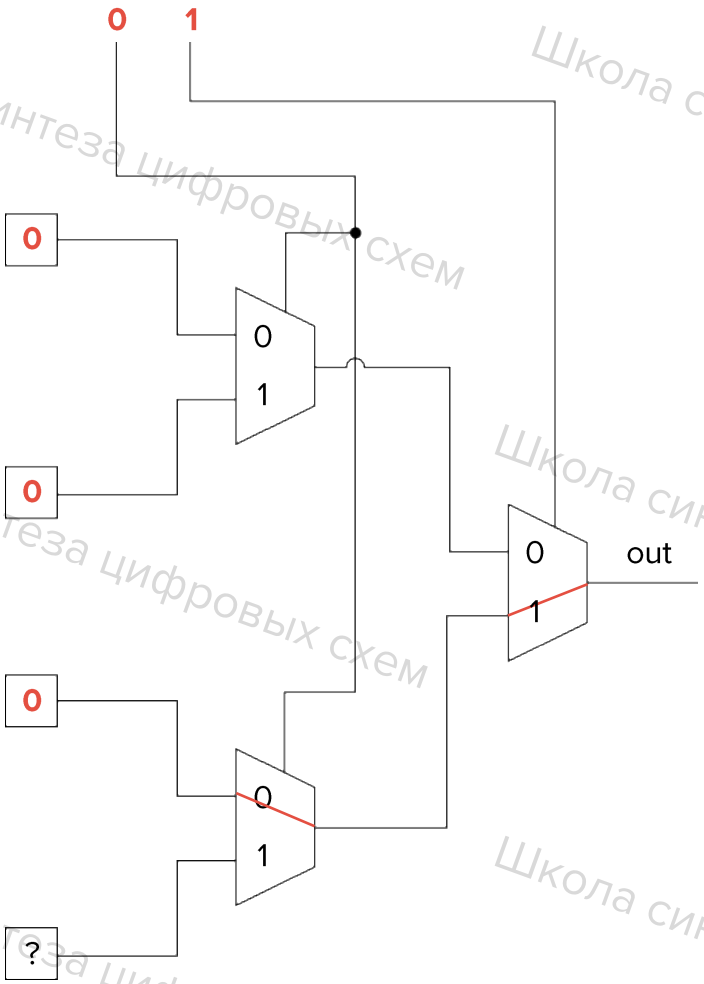
a	b	F (a, b)
0	0	0
0	1	
1	0	
1	1	

ЭЛЕМЕНТ «И» ЧЕРЕЗ LUT-2



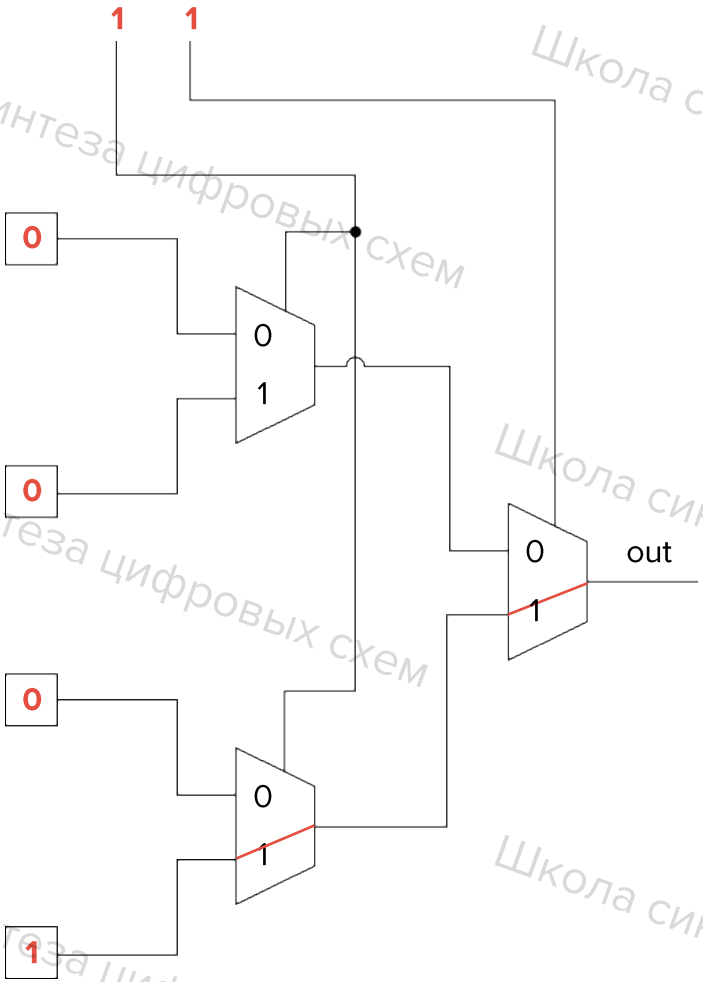
a	b	F (a, b)
0	0	0
0	1	0
1	0	
1	1	

ЭЛЕМЕНТ «И» ЧЕРЕЗ LUT-2



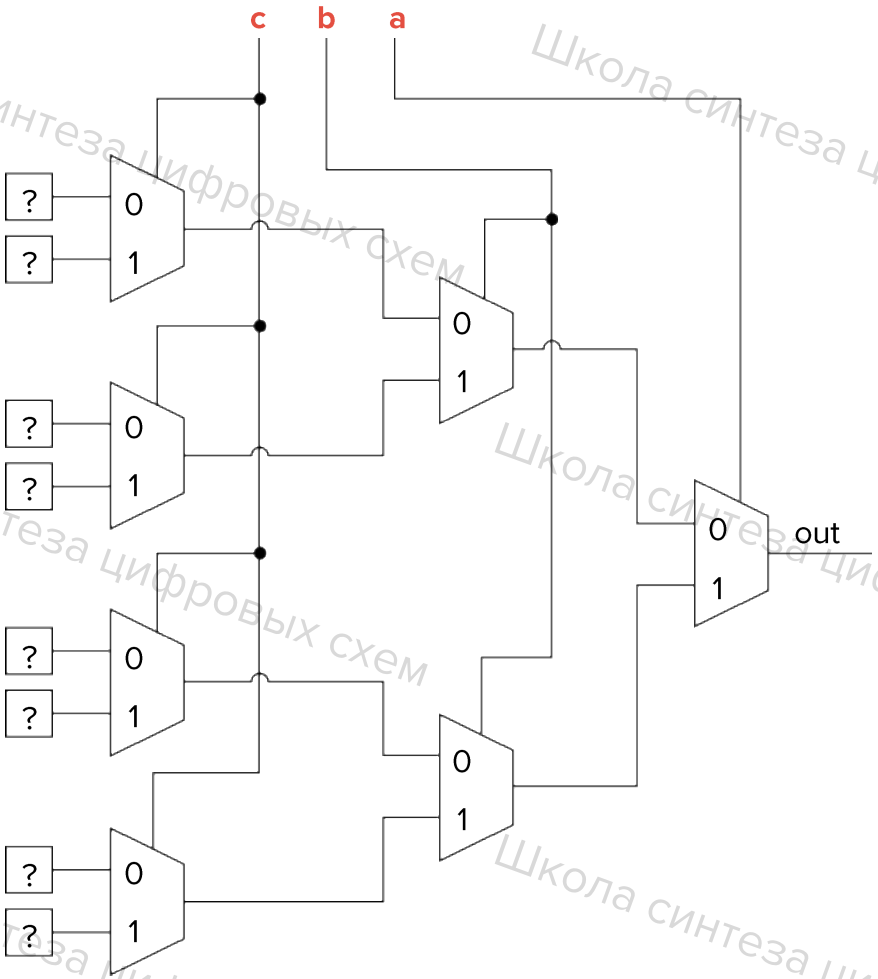
a	b	F (a, b)
0	0	0
0	1	0
1	0	0
1	1	

ЭЛЕМЕНТ «И» ЧЕРЕЗ LUT-2



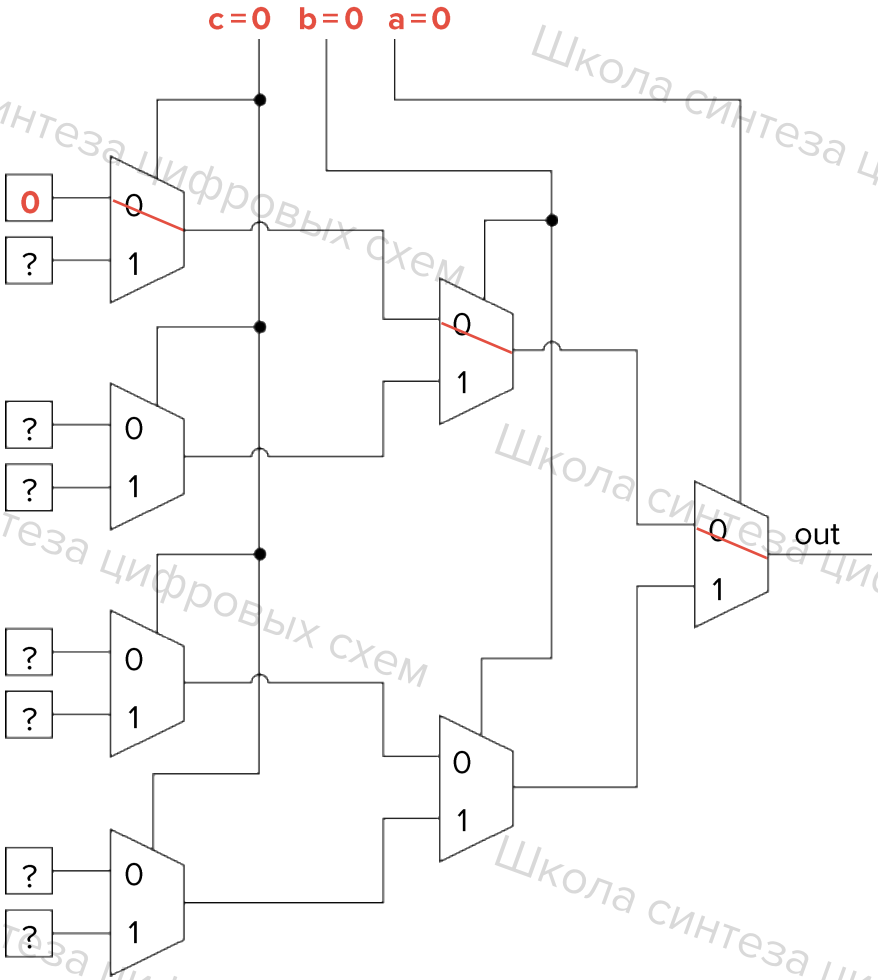
a	b	F (a, b)
0	0	0
0	1	0
1	0	0
1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



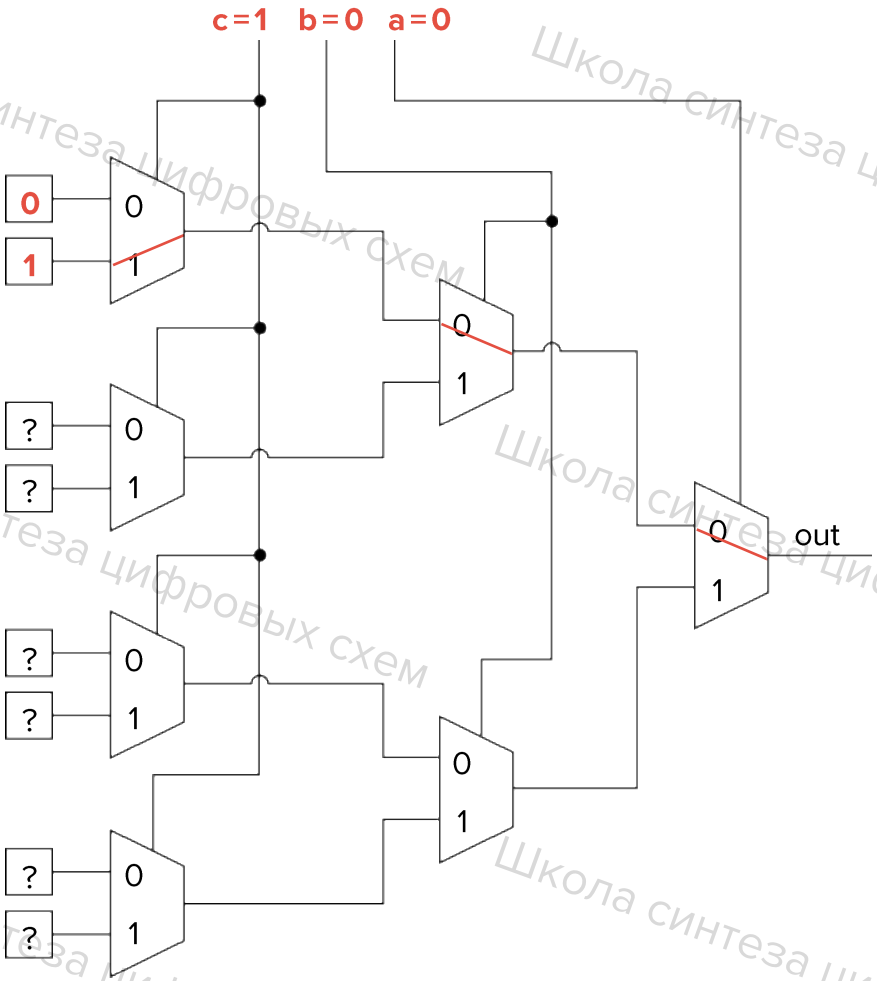
a	b	c	out = a ^ b ^ c
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



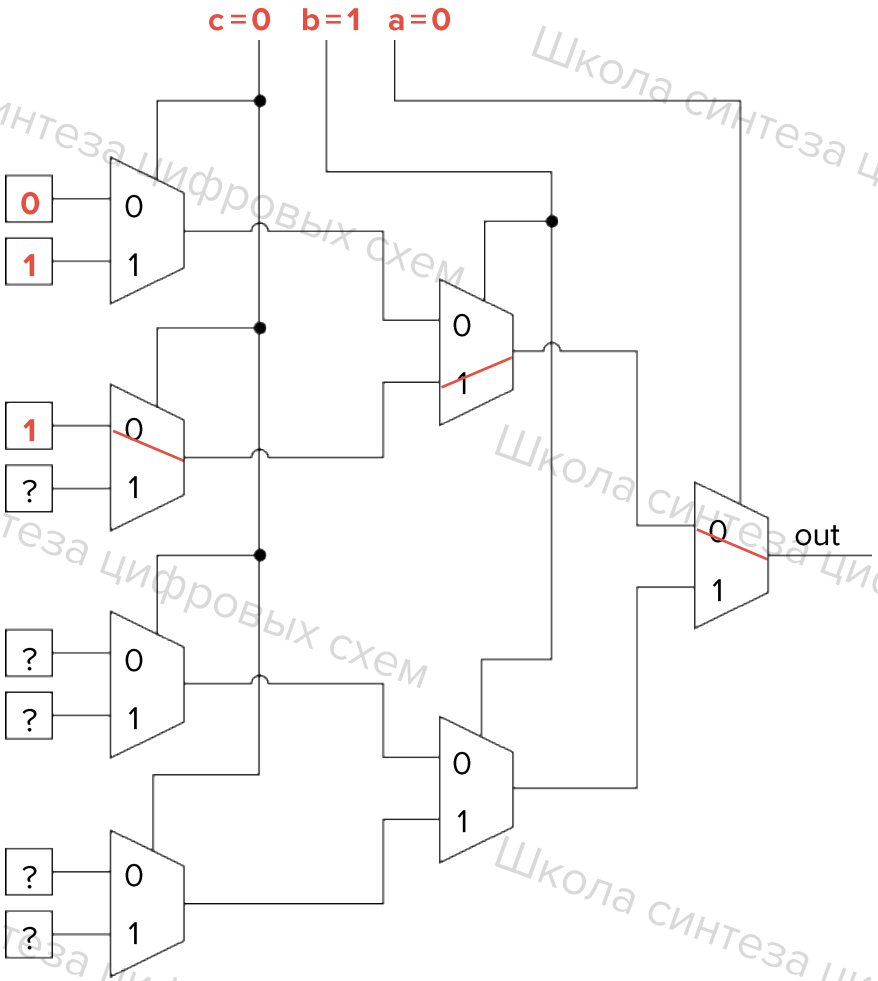
a	b	c	out = a ^ b ^ c
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



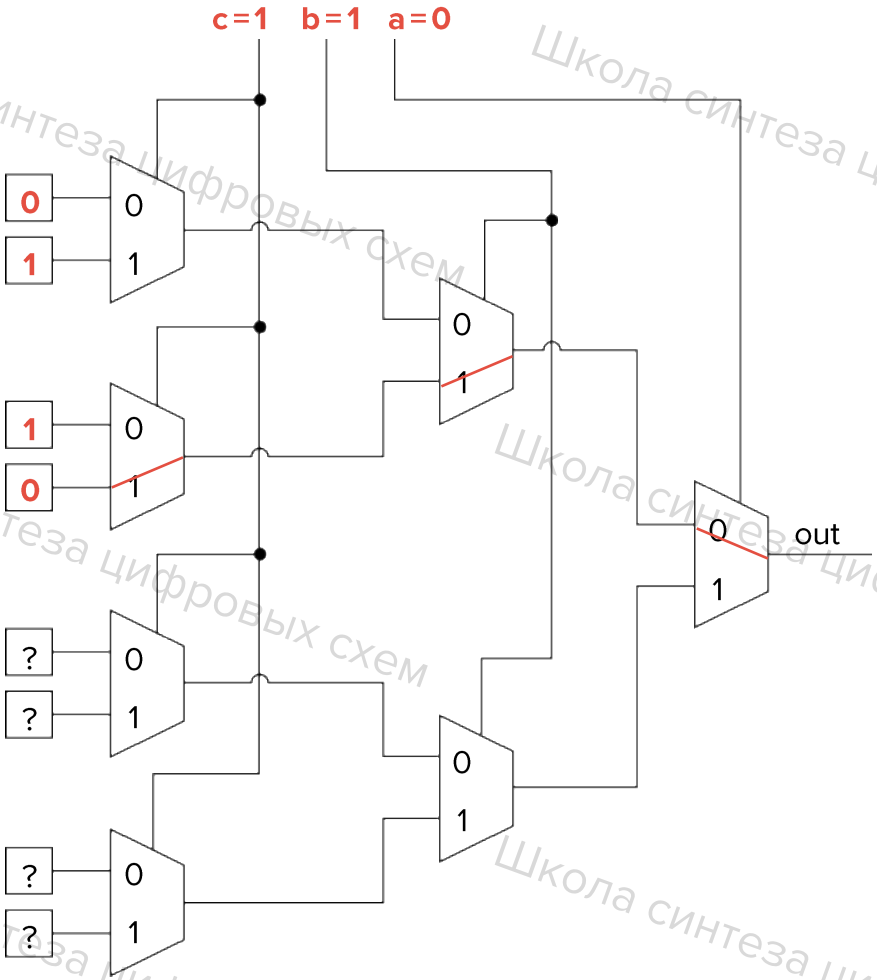
a	b	c	out = a ^ b ^ c
0	0	0	0
0	**0**	**1**	**1**
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



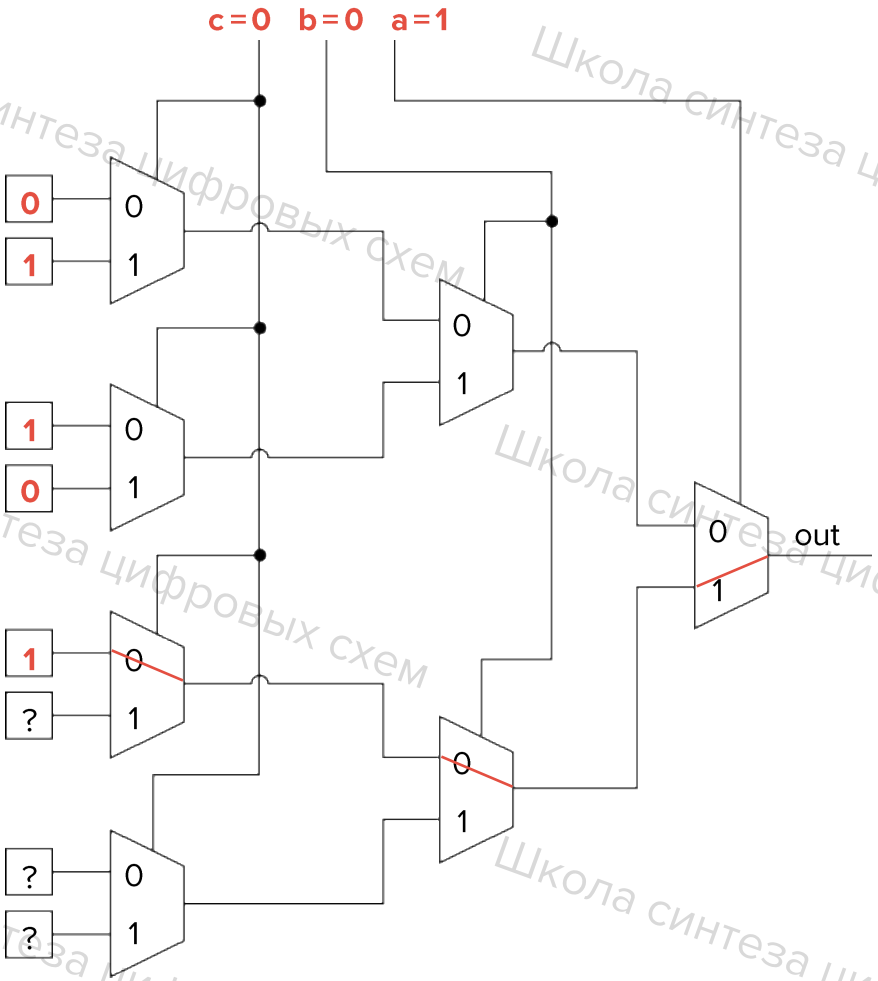
a	b	c	out = a ^ b ^ c
0	0	0	0
0	0	1	1
0	**1**	**0**	**1**
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



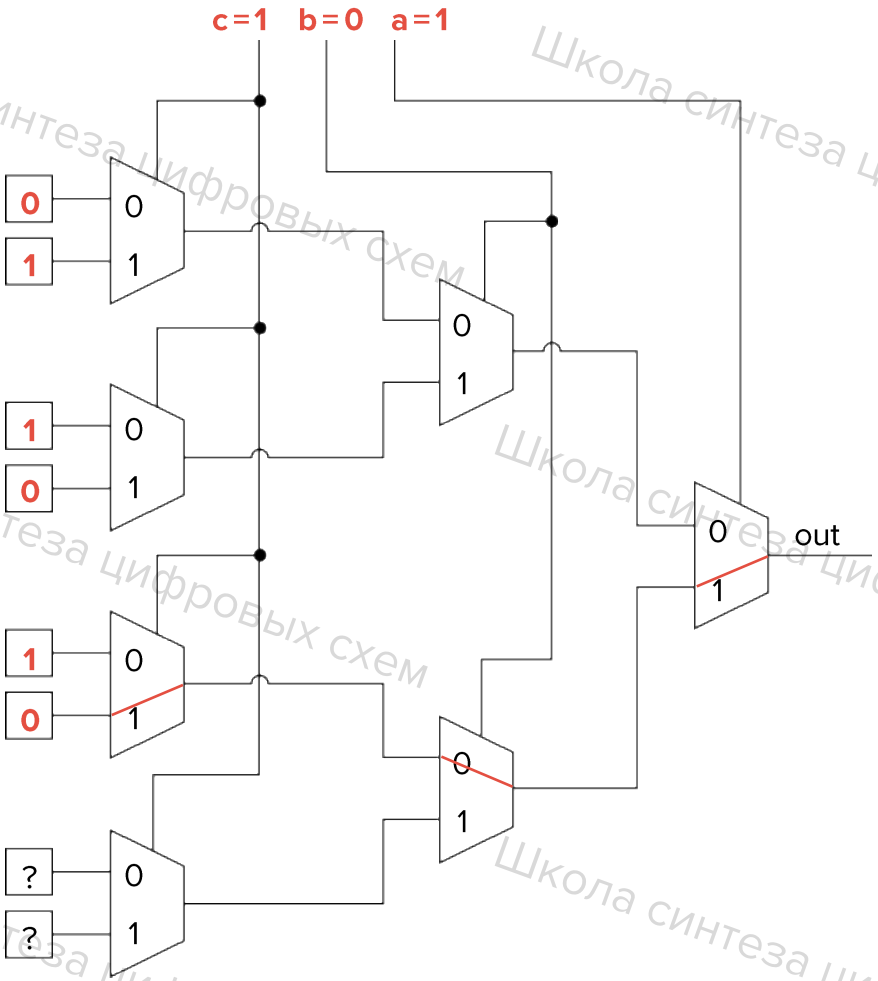
a	b	c	out = a ^ b ^ c
0	0	0	0
0	0	1	1
0	1	0	1
0	**1**	**1**	**0**
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



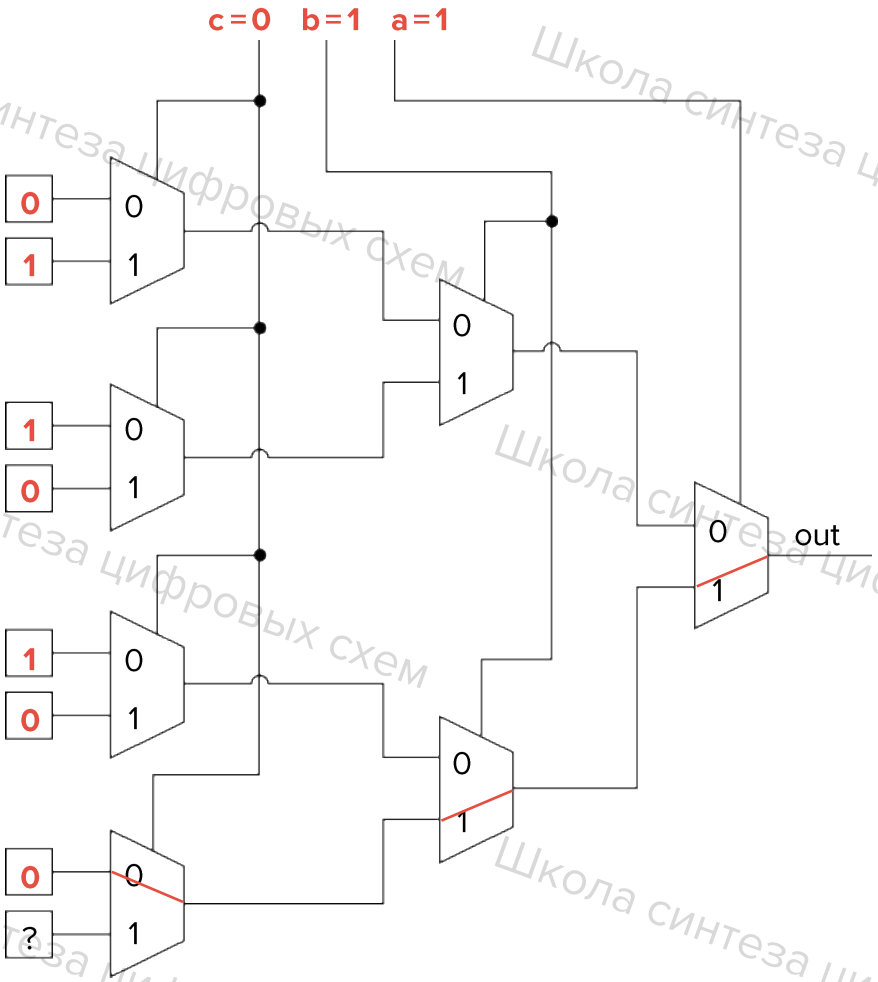
a	b	c	out = a ^ b ^ c
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	**0**	**0**	**1**
1	0	1	0
1	1	0	0
1	1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



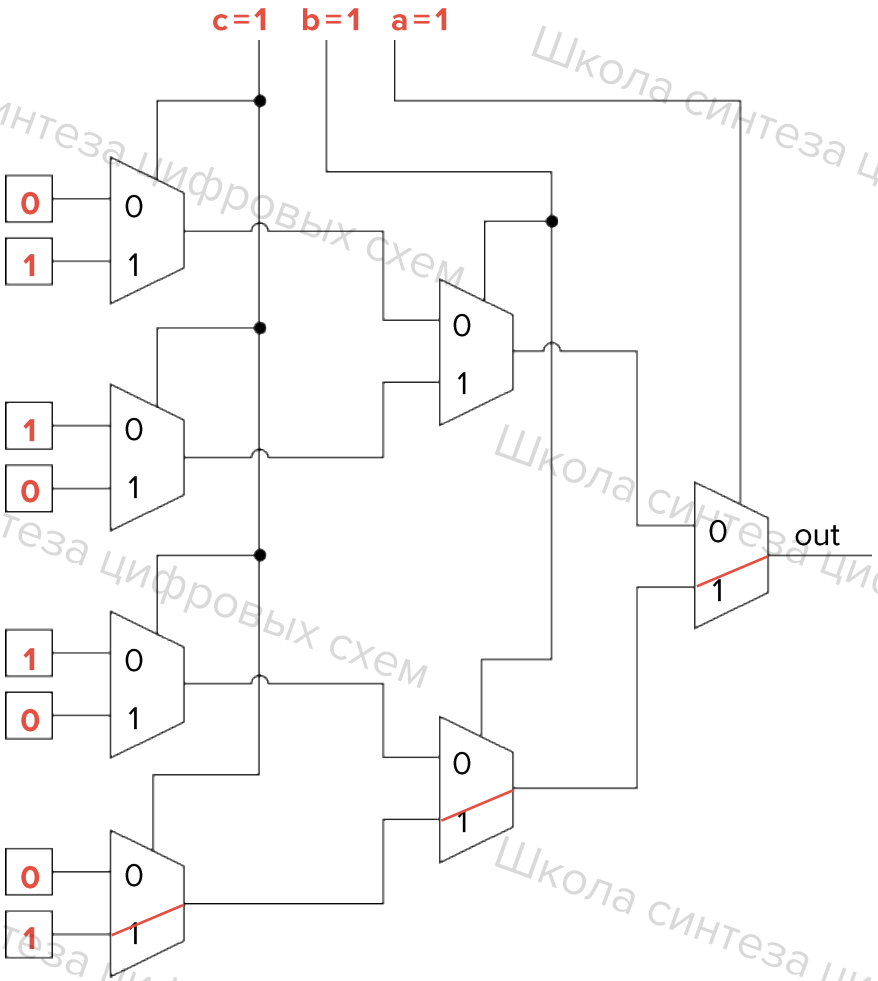
a	b	c	out = a ^ b ^ c
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	**0**	**1**	**0**
1	1	0	0
1	1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



a	b	c	out = a ^ b ^ c
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	**1**	**0**	**0**
1	1	1	1

РЕАЛИЗАЦИЯ «A ^ B ^ C» ЧЕРЕЗ LUT-3



a	b	c	out = a ^ b ^ c
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	**1**	**1**	**1**

ПРО ПЕРЕВОД RTL В FPGA


```
module counter_8_bit (
    input               clk,
    input               rst,
    output logic [7:0] num
);

always_ff @(posedge clk) begin
    if (rst) num <= '0;
    else num <= num + 1;
end

endmodule
```

Прошивание FPGA

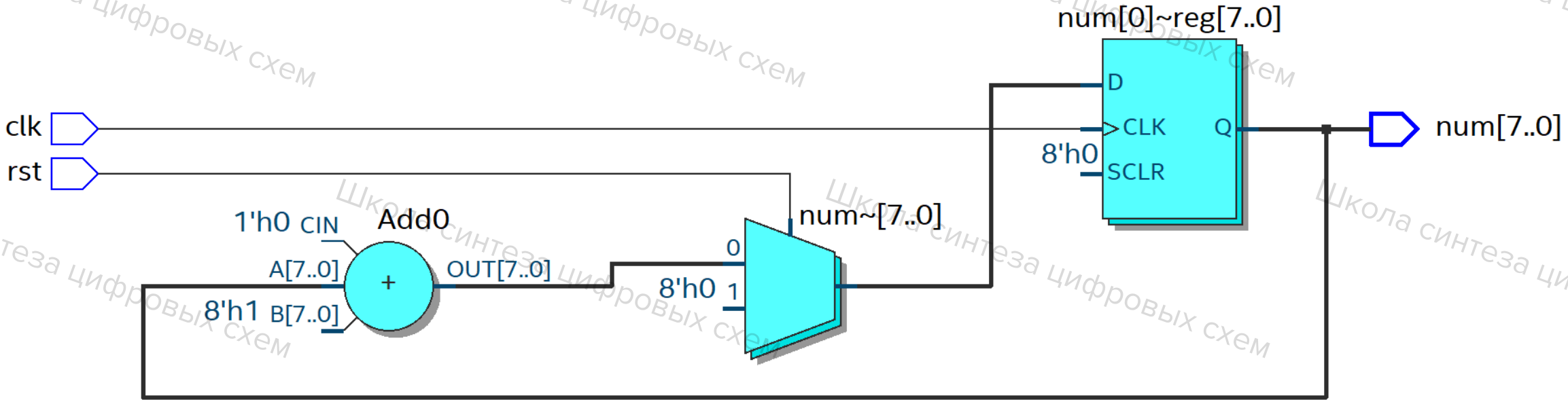
Содержит в себе пункты аналогичные маршруту RTL to GDS, хотя они и являются практически полностью автоматизированными

В качестве примера рассмотрим перевод 8-битного счётчика в FPGA

Школа синтеза цифровых схем

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

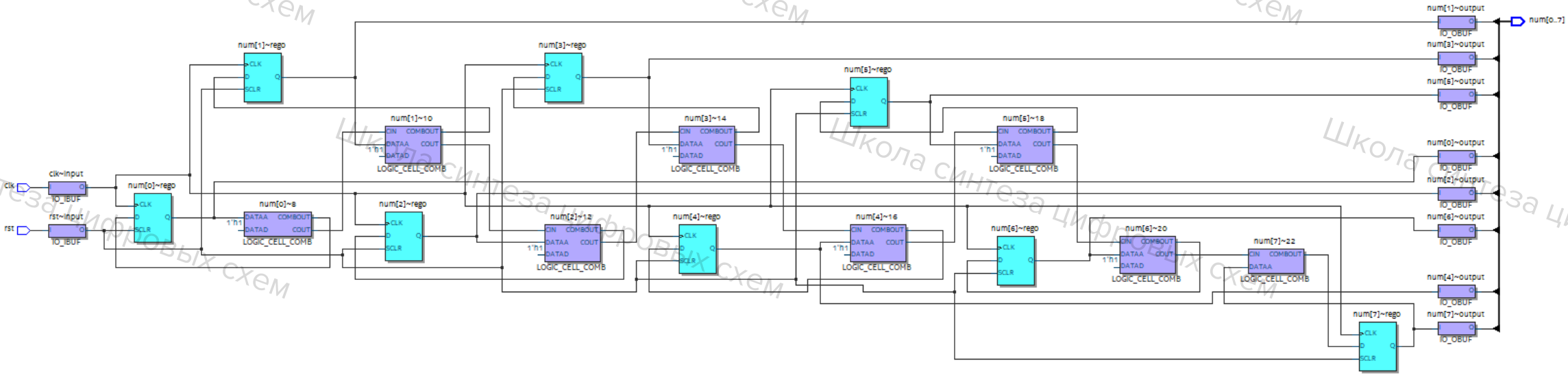
RTL VIEWER (АНАЛОГ GENERIC)



Школа синтеза цифровых схем

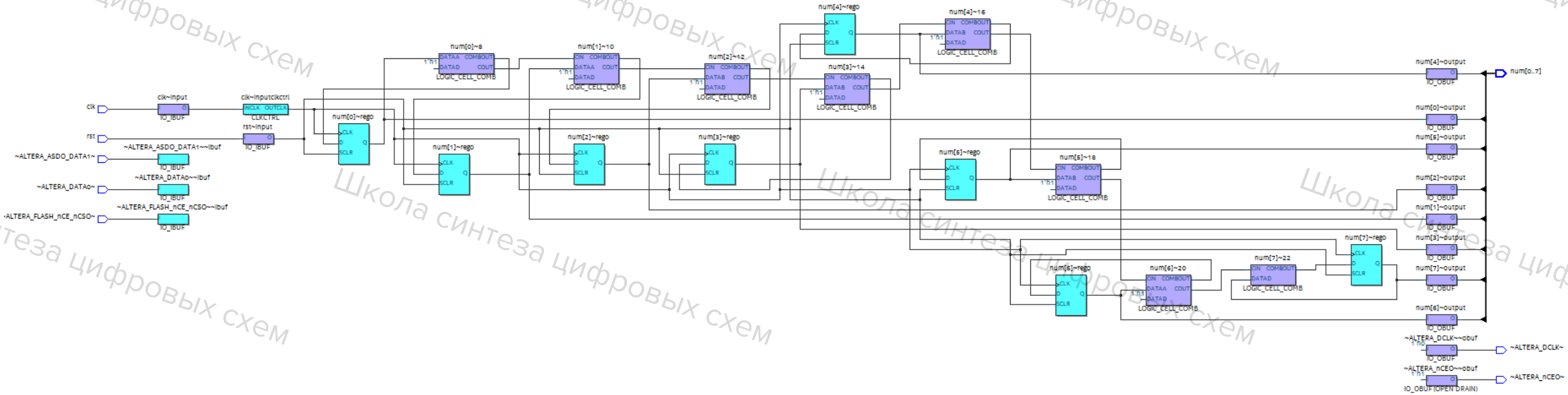
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

TECHNOLOGY MAP VIEWER (ПОСЛЕ MAPPING)



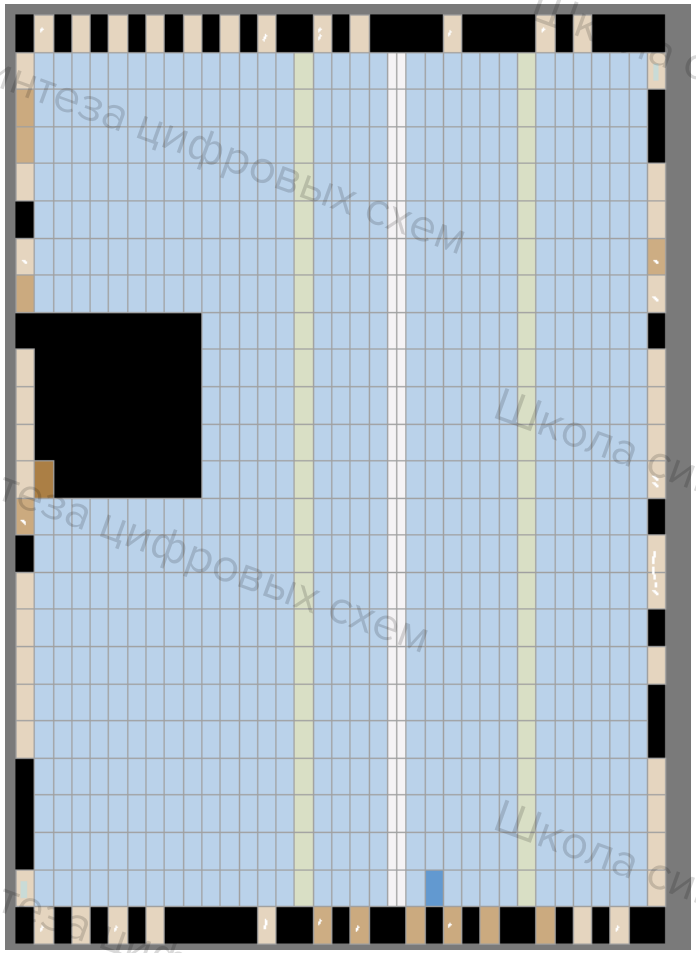
школа синтеза цифровых схем

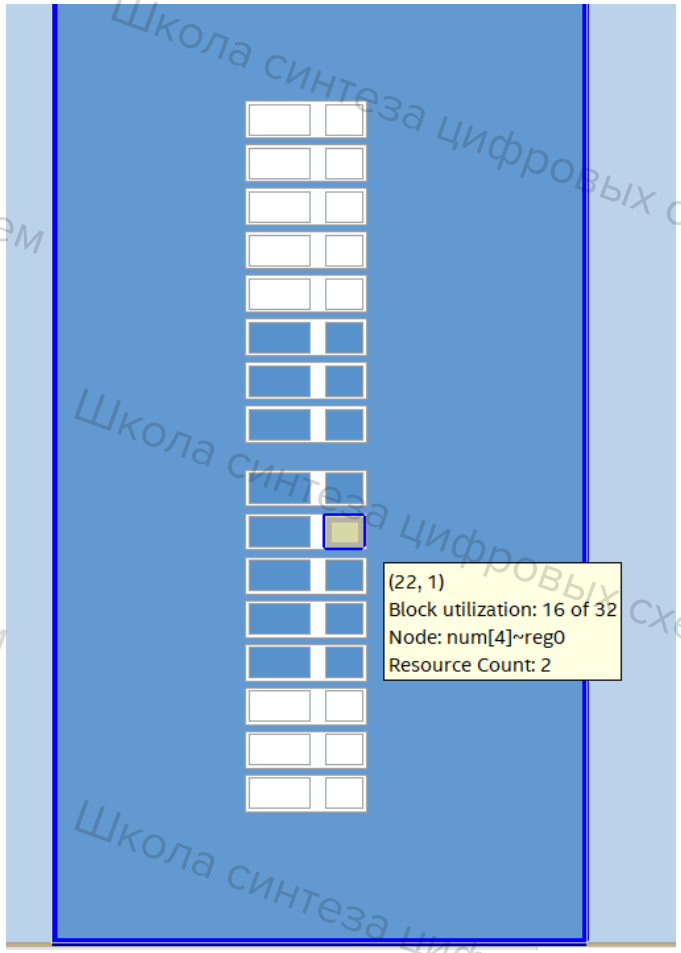
TECHNOLOGY MAP VIEWER (ПОСЛЕ ОПТИМИЗАЦИИ)



ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

РАЗМЕЩЕНИЕ В FPGA (АНАЛОГ ТОПОЛОГИИ)

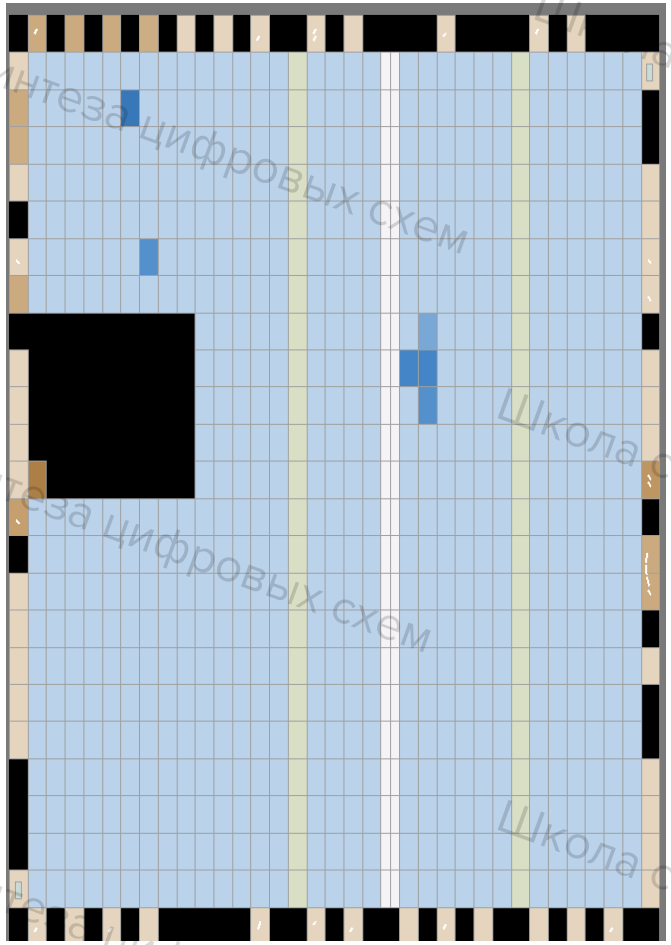


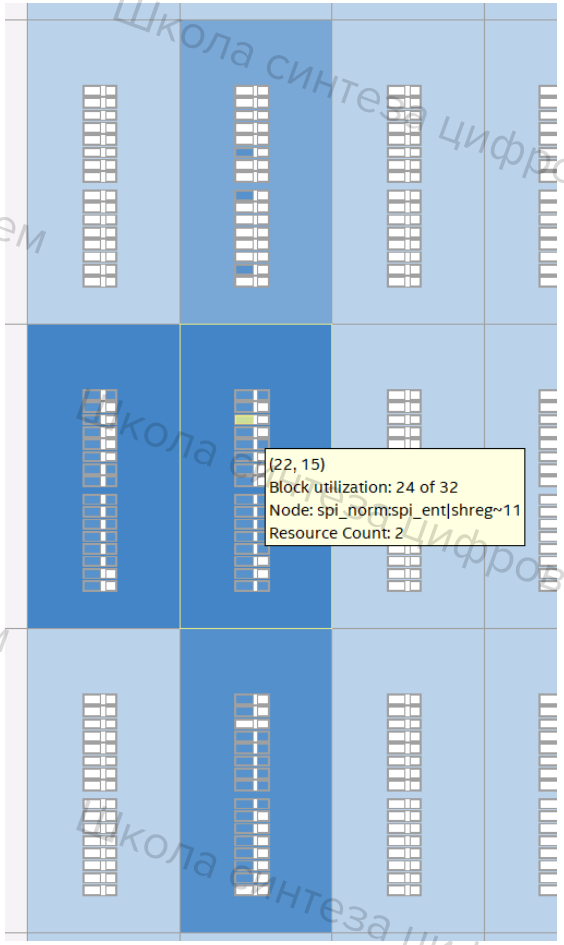


Школа синтеза цифровых схем

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

РАЗМЕЩЕНИЕ БОЛЕЕ СЛОЖНОЙ СХЕМЫ В FPGA



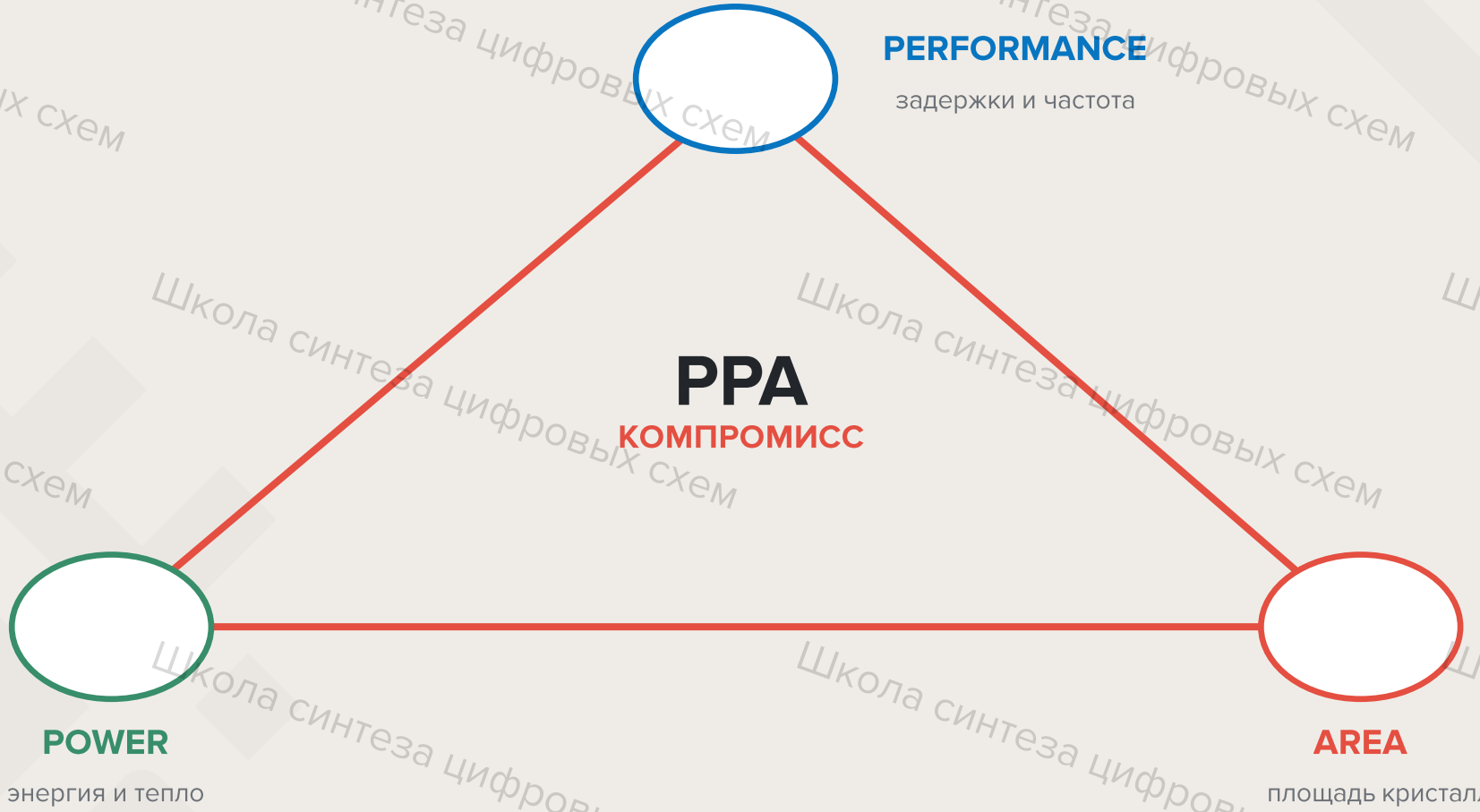


Школа синтеза цифровых схем

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

PPA: БАЗОВЫЙ КОМПРОМИСС ПРОЕКТИРОВАНИЯ ЦИФРОВЫХ СХЕМ

Power • Performance • Area





Форма обратной связи

Авторы (разработчики материала) лекции – Силантьев Александр Михайлович, Томащук Юрий Михайлович, Старунова Анна Сергеевна и Терехин Владислав Владимирович.

Использование материалов и записи лекции и/или их частей без предварительного согласия не допускается.

По вопросам использования материалов и записи лекции в коммерческих целях необходимо направить обращение в ООО «КНС ГРУПП» (YADRO) по адресу электронной почты synthesis@yadro.com.

По вопросам некоммерческого использования материалов и записи лекции обращение может быть направлено в ООО «КНС ГРУПП» (YADRO) по адресу электронной почты synthesis@yadro.com, либо на адреса электронной почты автора Силантьева А.М. silantiev@org.miet.ru и Терехина В.В. Terekhinvladislav@gmail.com. Такое обращение обязательно должно содержать описание цели использования.

Школа синтеза цифровых схем